Aprendizaje por Refuerzo
Métodos Aproximados: DRL
(Aprendizaje por Refuerzo Profundo)

Manuel Sánchez-Montañés

Q-Learning: limitaciones

Q-learning: limitaciones

- Se basa en construir la tabla Q(s,a), que valora “cómo de bueno” es realizar la acción a desde el estado s

	a0	a1	a2
S0	0.1	1	0.5
S1	0.95	-0.2	0.3
S2	1.5	2	-1
S3	-0.2	0.1	0.4
S4	1.3	-1.1	0.2

Q-learning: limitaciones

- Se basa en construir la tabla Q(s,a), que valora “cómo de bueno” es realizar la acción a desde el estado s

	a0	a1	a2
S0	0.1	1	0.5
S1	0.95	-0.2	0.3
S2	1.5	2	-1
S3	-0.2	0.1	0.4
S4	1.3	-1.1	0.2

- Después de construir la tabla, si se ha aprendido correctamente, la política óptima coincide con elegir en cada s la acción con mayor Q

Q-learning: limitaciones

- Se basa en construir la tabla Q(s,a), que valora “cómo de bueno” es realizar la acción a desde el estado s

	a0	a1	a2
S0	0.1	1	0.5
S1	0.95	-0.2	0.3
S2	1.5	2	-1
S3	-0.2	0.1	0.4
S4	1.3	-1.1	0.2

- Después de construir la tabla, si se ha aprendido correctamente, la política óptima coincide con elegir en cada s la acción con mayor Q

Q-learning: limitaciones

- Se va construyendo la tabla **Q**, que tiene tantas filas como estados posibles, y tantas columnas como acciones posibles.

Q-learning: limitaciones

- Se va construyendo la tabla **Q**, que tiene tantas filas como estados posibles, y tantas columnas como acciones posibles.
- Si desde el estado $\mathbf{s}_t$ se realiza una acción a_t y esta da lugar a un refuerzo $\mathbf{r}_t$ y una transición al estado $\mathbf{s}_{t+1}$:

Q-learning: limitaciones

- Se va construyendo la tabla **Q**, que tiene tantas filas como estados posibles, y tantas columnas como acciones posibles.
- Si desde el estado s_t se realiza una acción a_t y esta da lugar a un refuerzo r_t y una transición al estado s_{t+1} :

$$\Delta Q(s_t, a_t) = \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s, a)]$$

Q-learning: limitaciones

- Q-Learning construye una tabla

Q-learning: limitaciones

- Q-Learning construye una tabla
- ¿Qué hacer si queremos aplicar este método a un coche autónomo que tiene una cámara de 100x100 píxels en blanco y negro (256 niveles de gris)?

Q-learning: limitaciones

- Q-Learning construye una tabla
- ¿Qué hacer si queremos aplicar este método a un coche autónomo que tiene una cámara de 100x100 píxels en blanco y negro (256 niveles de gris)?
- Tamaño espacio de estados =256^100000 !!!

Q-learning: limitaciones

- Q-Learning construye una tabla
- ¿Qué hacer si queremos aplicar este método a un coche autónomo que tiene una cámara de 100x100 píxels en blanco y negro (256 niveles de gris)?
- Tamaño espacio de estados =256^100000 !!!
- No solamente hay un número enorme de estados, sino que seguramente cada uno de ellos se visitará 0 o 1 veces a lo sumo

Introducción al DRL (Deep Reinforcement Learning)

Motivación

- El Deep Learning puede aprender características end-to-end en problemas de alta dimensionalidad / grandes espacios de estados

Motivación

- El Deep Learning puede aprender características end-to-end en problemas de alta dimensionalidad / grandes espacios de estados

- Maldición de la dimensionalidad

Motivación

- El Deep Learning puede aprender características end-to-end en problemas de alta dimensionalidad / grandes espacios de estados

- Maldición de la dimensionalidad

- Las imágenes b/n con 256 niveles de gris y 100x100 píxeles definen un espacio con 256^10000 estados

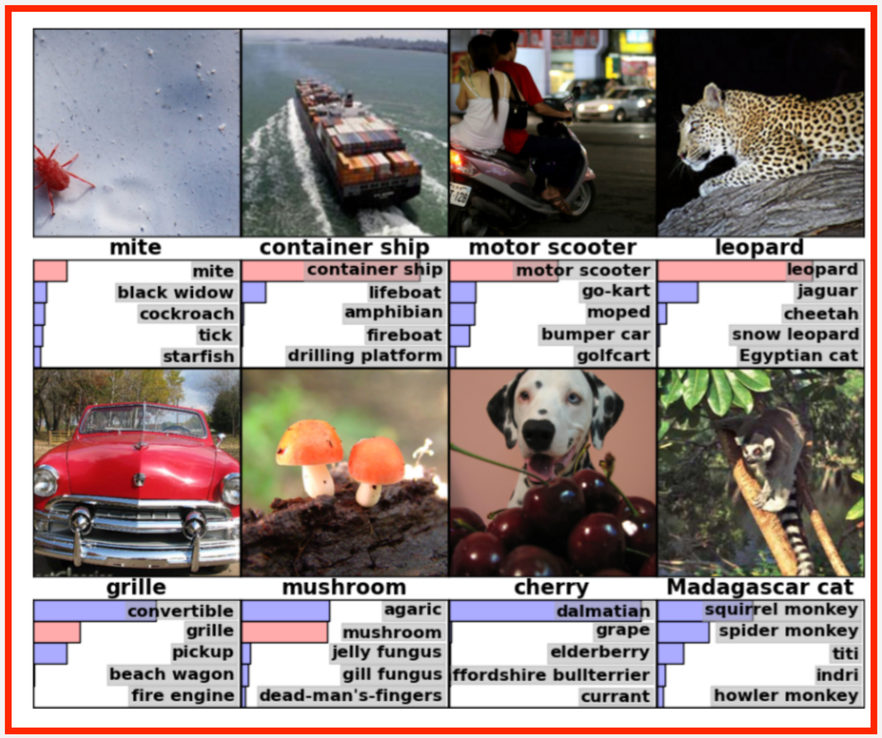
Motivación

- El Deep Learning puede aprender características end-to-end en problemas de alta dimensionalidad / grandes espacios de estados
- Maldición de la dimensionalidad
- Las imágenes b/n con 256 niveles de gris y 100x100 píxeles definen un espacio con 256^10000 estados
- ¿Cómo aprender? Nunca hay suficientes muestras para llenar el espacio de estados

Motivación

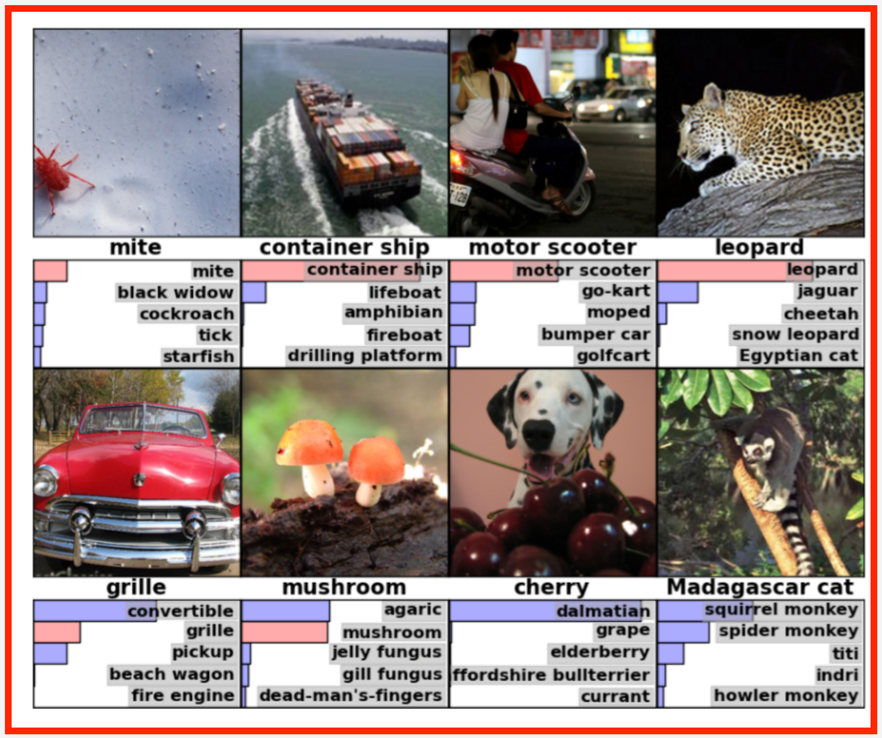
- El Deep Learning puede aprender características end-to-end en problemas de alta dimensionalidad / grandes espacios de estados
- Maldición de la dimensionalidad
- Las imágenes b/n con 256 niveles de gris y 100x100 píxeles definen un espacio con 256^10000 estados
- ¿Cómo aprender? Nunca hay suficientes muestras para llenar el espacio de estados
- Aproximar, Generalizar. Explotar la continuidad

Motivación



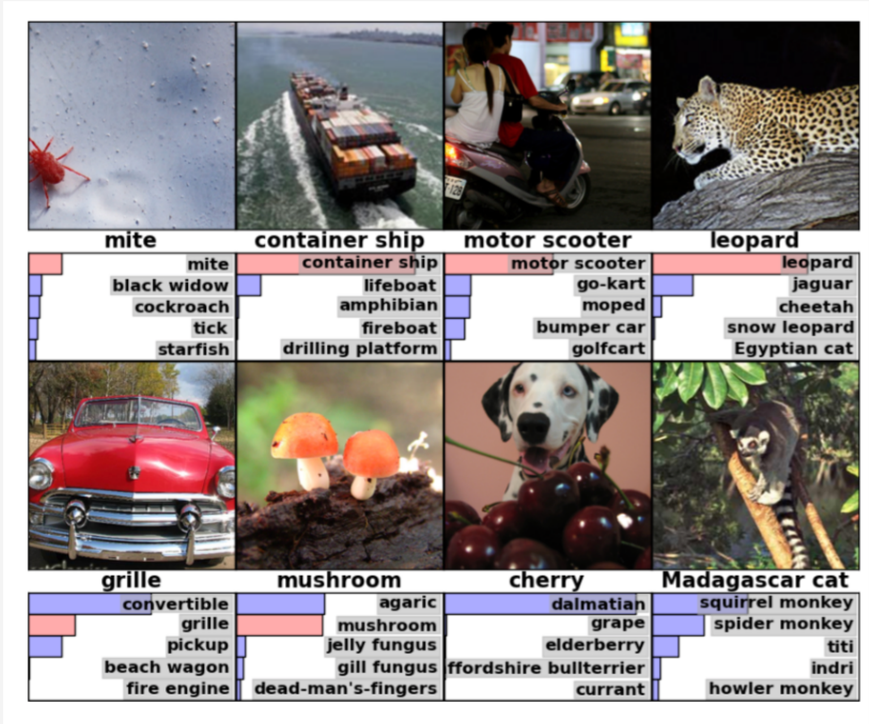
- En 2012 el DL supervisado triunfa en el ILSVRC (competición sobre ImageNet)

Motivación



- En 2012 el DL supervisado triunfa en el ILSVRC (competición sobre ImageNet)
- El DL supervisado se desarrolla en mucha parte gracias a ImageNet

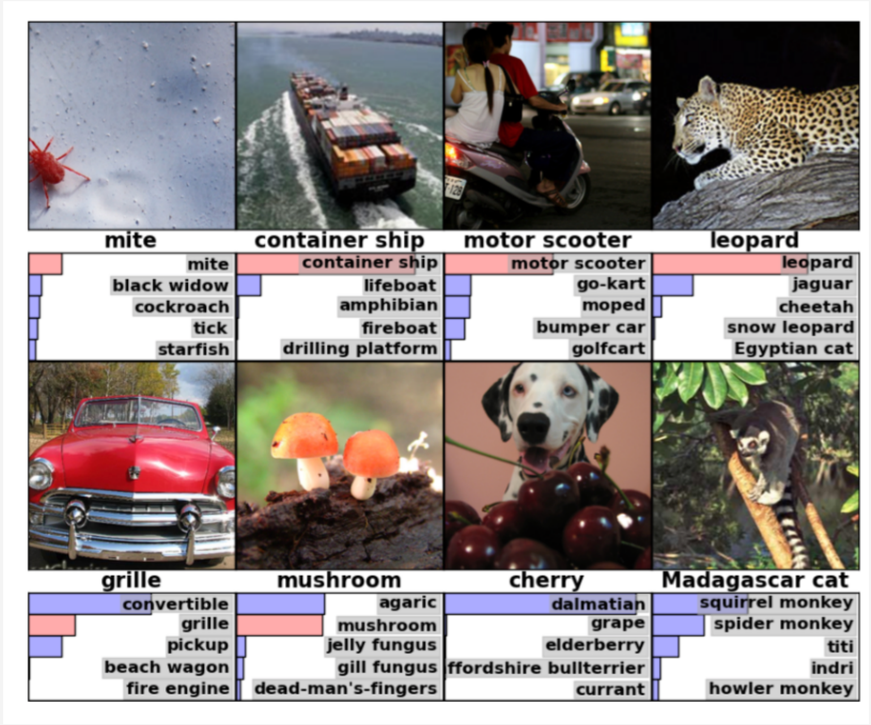
Motivación





- En 2012 el DL supervisado triunfa en el ILSVRC (competición sobre ImageNet)
- El DL supervisado se desarrolla en mucha parte gracias a ImageNet
- En RL, en ese año, el estado del arte en RL con redes neuronales es ~ TD-Gammon, de 1992. La red tiene 40 neuronas ocultas

Motivación





- En 2012 el DL supervisado triunfa en el ILSVRC (competición sobre ImageNet)
- El DL supervisado se desarrolla en mucha parte gracias a ImageNet
- En RL, en ese año, el estado del arte en RL con redes neuronales es ~ TD-Gammon, de 1992. La red tiene 40 neuronas ocultas
- ¿Qué podemos hacer con Deep CNN en RL?

Atari Learning Environment (ALE)



- El Aprendizaje por Refuerzo Profundo (Deep RL) se desarrolla en mucha parte gracias a ALE

Atari 2600



- Consola
- Cartucho de juego ROM 128 bytes RAM
- Entradas: píxeles
- Salida: acciones del joystick

Atari Learning Environment (ALE)

- Minh et al. 2013 / 2015



Atari Learning Environment (ALE)

- Minh et al. 2013 / 2015
- Un ordenador aprende a jugar a 6 consolas Atari de 1980



Atari Learning Environment (ALE)

- Minh et al. 2013 / 2015
- Un ordenador aprende a jugar a 6 consolas Atari de 1980
- End-to-end



Atari Learning Environment (ALE)

- Minh et al. 2013 / 2015
- Un ordenador aprende a jugar a 6 consolas Atari de 1980
- End-to-end
- De los píxeles al joystick



Atari Learning Environment (ALE)

- Minh et al. 2013 / 2015
- Un ordenador aprende a jugar a 6 consolas Atari de 1980
- End-to-end
- De los píxeles al joystick
- Emulador para consola



Deep Reinforcement Learning

LETTER

doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]

The theory of reinforcement learning provides a normative account[1], deeply rooted in psychological[2] and neuroscientific[3] perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems[4,5], the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms[3]. While reinforcement learning agents have achieved some successes in a variety of domains[6–8], their applicability has previously been limited to domains in which useful features can be handcrafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks[9–11] to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games[12]. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games tester across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

We set out to create a single algorithm that would be able to develop a wide range of competencies on a varied range of challenging tasks—a central goal of general artificial intelligence[13] that has eluded previous efforts[8,14,15]. To achieve this, we developed a novel agent, a deep Q-network (DQN), which is able to combine reinforcement learning with a class of artificial neural network[16] known as deep neural networks. Notably, recent advances in deep neural networks[9–11], in which several layers of nodes are used to build up progressively more abstract representations of the data, have made it possible for artificial neural networks to learn concepts such as object categories directly from raw sensory data. We use one particularly successful architecture, the deep convolutional network[17], which uses hierarchical layers of tiled convolutional filters to mimic the effects of receptive fields—inspired by Hubel and Wiesel's seminal work on feedforward processing in early visual cortex[18]—thereby exploiting the local spatial correlations present in images, and building in robustness to natural transformations such as changes of viewpoint or scale.

We consider tasks in which the agent interacts with an environment through a sequence of observations, actions and rewards. The goal of the agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s,a) = \max_\pi \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots | s_t = s,\, a_t = a,\, \pi\right],$$

which is the maximum sum of rewards r_t discounted by γ at each time-step t, achievable by a behaviour policy $\pi = P(a|s)$, after making an observation (s) and taking an action (a) (see Methods)[19].

Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q) function[20]. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy and therefore change the data distribution, and the correlations between the action-values (Q) and the target values $r + \gamma \max_{a'} Q(s', a')$. We address these instabilities with a novel variant of Q-learning, which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay[21–23] that randomizes over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action-values (Q) towards target values that are only periodically updated, thereby reducing correlations with the target.

While other stable methods exist for training neural networks in the reinforcement learning setting, such as neural fitted Q-iteration[24], these methods involve the repeated training of networks *de novo* on hundreds of iterations. Consequently, these methods, unlike our algorithm, are too inefficient to be used successfully with large neural networks. We parameterize an approximate value function $Q(s,a;\theta_i)$ using the deep convolutional neural network shown in Fig. 1, in which θ_i are the parameters (that is, weights) of the Q-network at iteration i. To perform experience replay we store the agent's experiences $e_t = (s_t, a_t, r_t, s_{t+1})$ at each time-step t in a data set $D_t = \{e_1, \ldots, e_t\}$. During learning, we apply Q-learning updates, on samples (or minibatches) of experience $(s,a,r,s') \sim U(D)$, drawn uniformly at random from the pool of stored samples. The Q-learning update at iteration i uses the following loss function:

for each minibatch?

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim U(D)}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s,a;\theta_i)\right)^2\right]$$

in which γ is the discount factor determining the agent's horizon, θ_i are the parameters of the Q-network at iteration i and θ_i^- are the network parameters used to compute the target at iteration i. The target network parameters θ_i^- are only updated with the Q-network parameters (θ_i) every C steps and are held fixed between individual updates (see Methods).

To evaluate our DQN agent, we took advantage of the Atari 2600 platform, which offers a diverse array of tasks ($n = 49$) designed to be

[1]Google DeepMind, 5 New Street Square, London EC4A 3TW, UK.
*These authors contributed equally to this work.

©2015 Macmillan Publishers Limited. All rights reserved

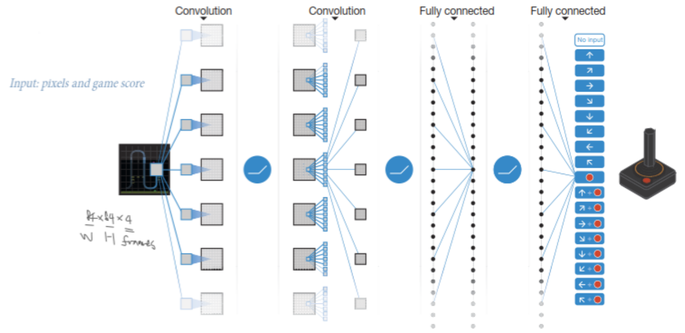


Figure 1 | Schematic illustration of the convolutional neural network. The details of the architecture are explained in the Methods. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ, followed by three convolutional layers (note: snaking blue line symbolizes sliding of each filter across input image) and two fully connected layers with a single output for each valid action. Each hidden layer is followed by a rectifier nonlinearity (that is, $\max(0,x)$).

difficult and engaging for human players. We used the same network architecture, hyperparameter values (see Extended Data Table 1) and learning procedure throughout—taking high-dimensional data (210×160 colour video at 60 Hz) as input—to demonstrate that our approach robustly learns successful policies over a variety of games based solely on sensory inputs with only very minimal prior knowledge (that is, merely the input data were visual images, and the number of actions available in each game, but not their correspondences; see Methods). Notably, our method was able to train large neural networks using a reinforcement learning signal and stochastic gradient descent in a stable manner—illustrated by the temporal evolution of two indices of learning (the agent's average score-per-episode and average predicted Q-values; see Fig. 2 and Supplementary Discussion for details).

We compared DQN with the best performing methods from the reinforcement learning literature on the 49 games where results were available[12,15]. In addition to the learned agents, we also report scores for a professional human games tester playing under controlled conditions and a policy that selects actions uniformly at random (Extended Data Table 2 and Fig. 3, denoted by 100% (human) and 0% (random) on y axis; see Methods). Our DQN method outperforms the best existing reinforcement learning methods on 43 of the games without incorporating any of the additional prior knowledge about Atari 2600 games used by other approaches (for example, refs 12, 15). Furthermore, our DQN agent performed at a level that was comparable to that of a professional human games tester across the set of 49 games, achieving more than 75% of the human score on more than half of the games (29 games;

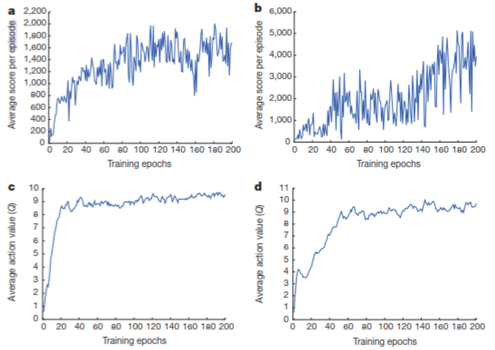


Figure 2 | Training curves tracking the agent's average score and average predicted action-value. a, Each point is the average score achieved per episode after the agent is run with ε-greedy policy ($\varepsilon = 0.05$) for 520 k frames on Space Invaders. **b**, Average score achieved per episode for Seaquest. **c**, Average predicted action-value on a held-out set of states on Space Invaders. Each point on the curve is the average of the action-value Q computed over the held-out set of states. Note that Q-values are scaled due to clipping of rewards (see Methods). **d**, Average predicted action-value on Seaquest. See Supplementary Discussion for details.

©2015 Macmillan Publishers Limited. All rights reserved

Deep Reinforcement Learning







Google DeepMind's
Deep Q-learning

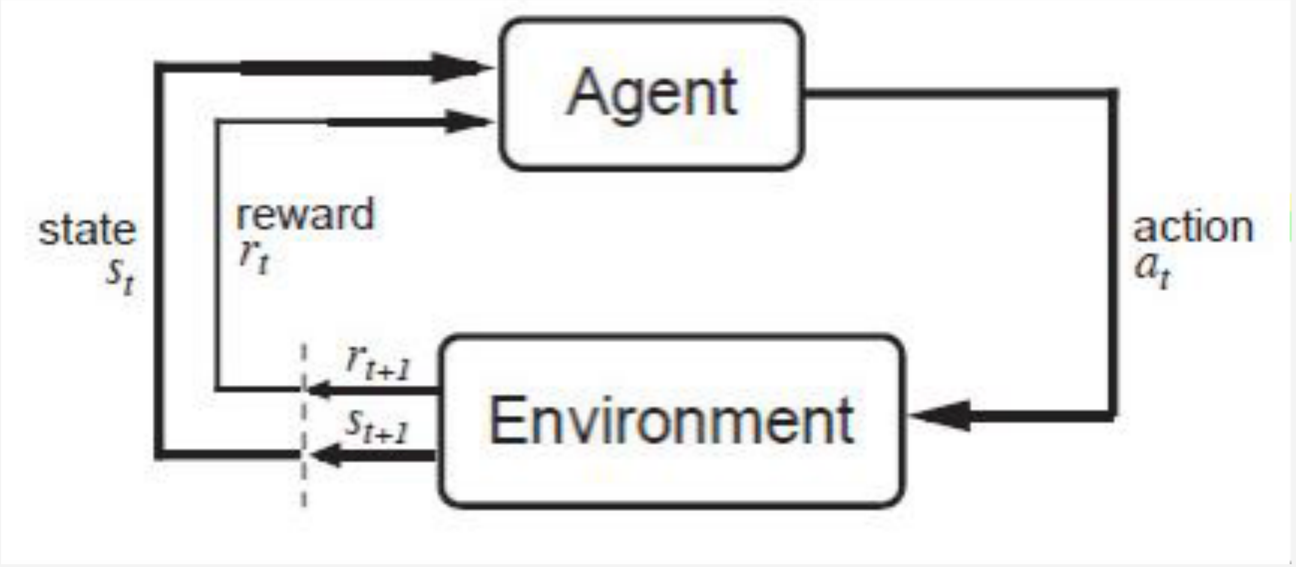
The algorithm will play Atari breakout.

The most important thing to know is that all the agent is given is sensory input (what you see on the screen) and it was ordered to maximize the score on the screen.

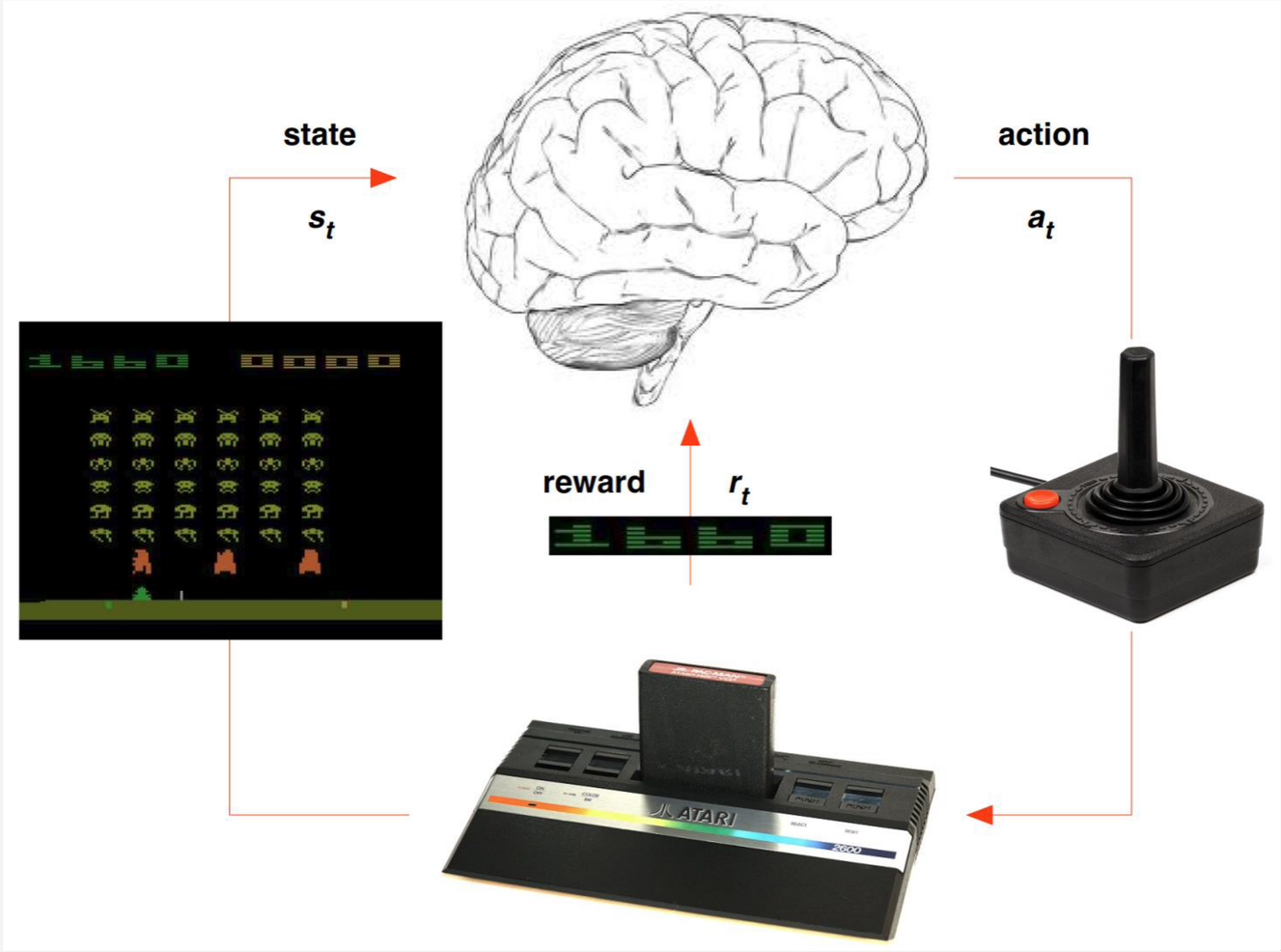
No domain knowledge is involved! This means that the algorithm doesn't know the concept of a ball or what the controls exactly do.

https://www.youtube.com/watch?v=V1eYniJ0Rnk

Deep Reinforcement Learning

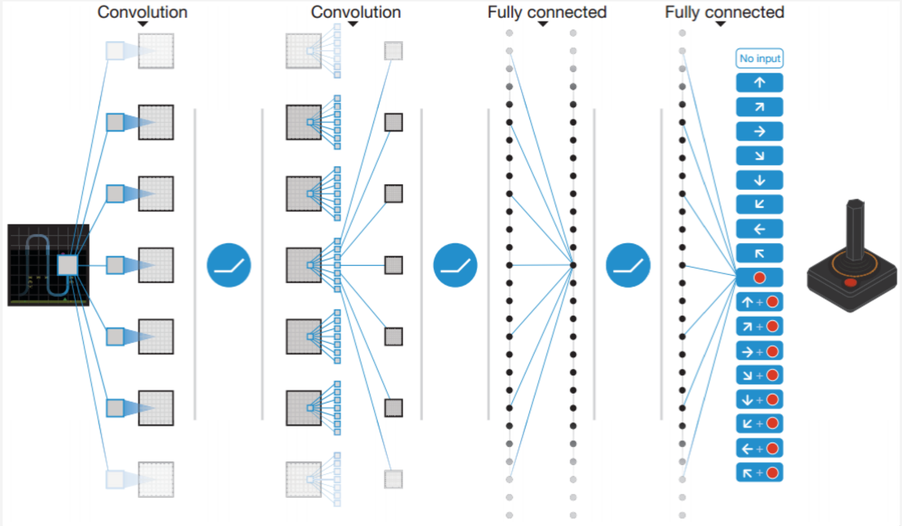


Deep Reinforcement Learning



Deep Reinforcement Learning

Arquitectura de la red CNN

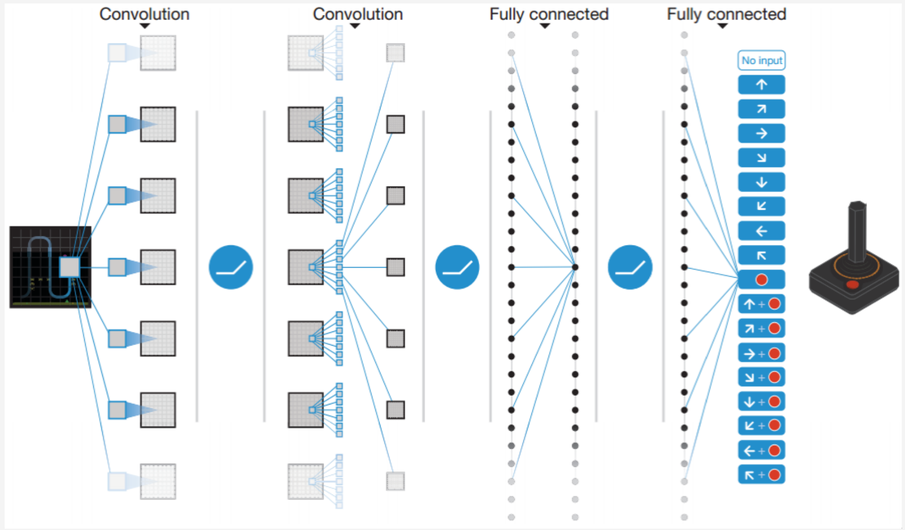


Layer	Input	Filter size	Stride	Num filters	Activation	Output
conv1	84x84x4	8x8	4	32	ReLU	20x20x32
conv2	20x20x32	4x4	2	64	ReLU	9x9x64
conv3	9x9x64	3x3	1	64	ReLU	7x7x64
fc4	7x7x64			512	ReLU	512
fc5	512			18	Linear	18

Deep Reinforcement Learning

Arquitectura de la red CNN

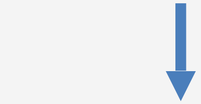
La entrada son 4 frames de 84x84 píxels (movimiento)



Layer	Input	Filter size	Stride	Num filters	Activation	Output
conv1	84x84x4	8x8	4	32	ReLU	20x20x32
conv2	20x20x32	4x4	2	64	ReLU	9x9x64
conv3	9x9x64	3x3	1	64	ReLU	7x7x64
fc4	7x7x64			512	ReLU	512
fc5	512			18	Linear	18

Deep Reinforcement Learning

datos observados en estado



Deep NN



$Q(s, a_1)$	$Q(s, a_2)$	...	$Q(s, a_m)$

$$loss = \left(\underbrace{r_t + \gamma \max_{a`} \hat{Q}(s_{t+1}, a`)}_{\text{Target}} - \underbrace{Q(s_t, a_t)}_{\text{Prediction}} \right)^2$$

Reward

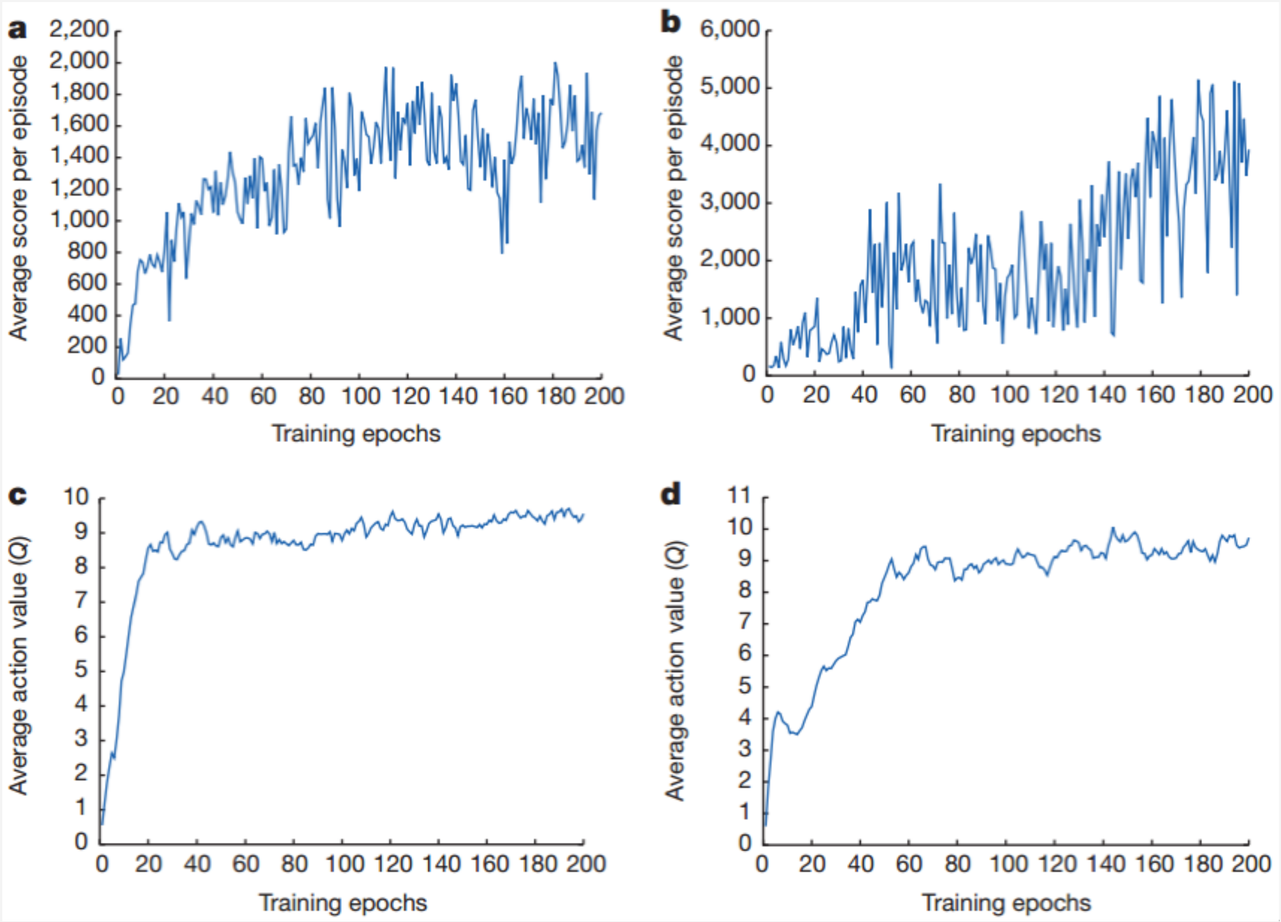
Decay Rate

Remember

Replay

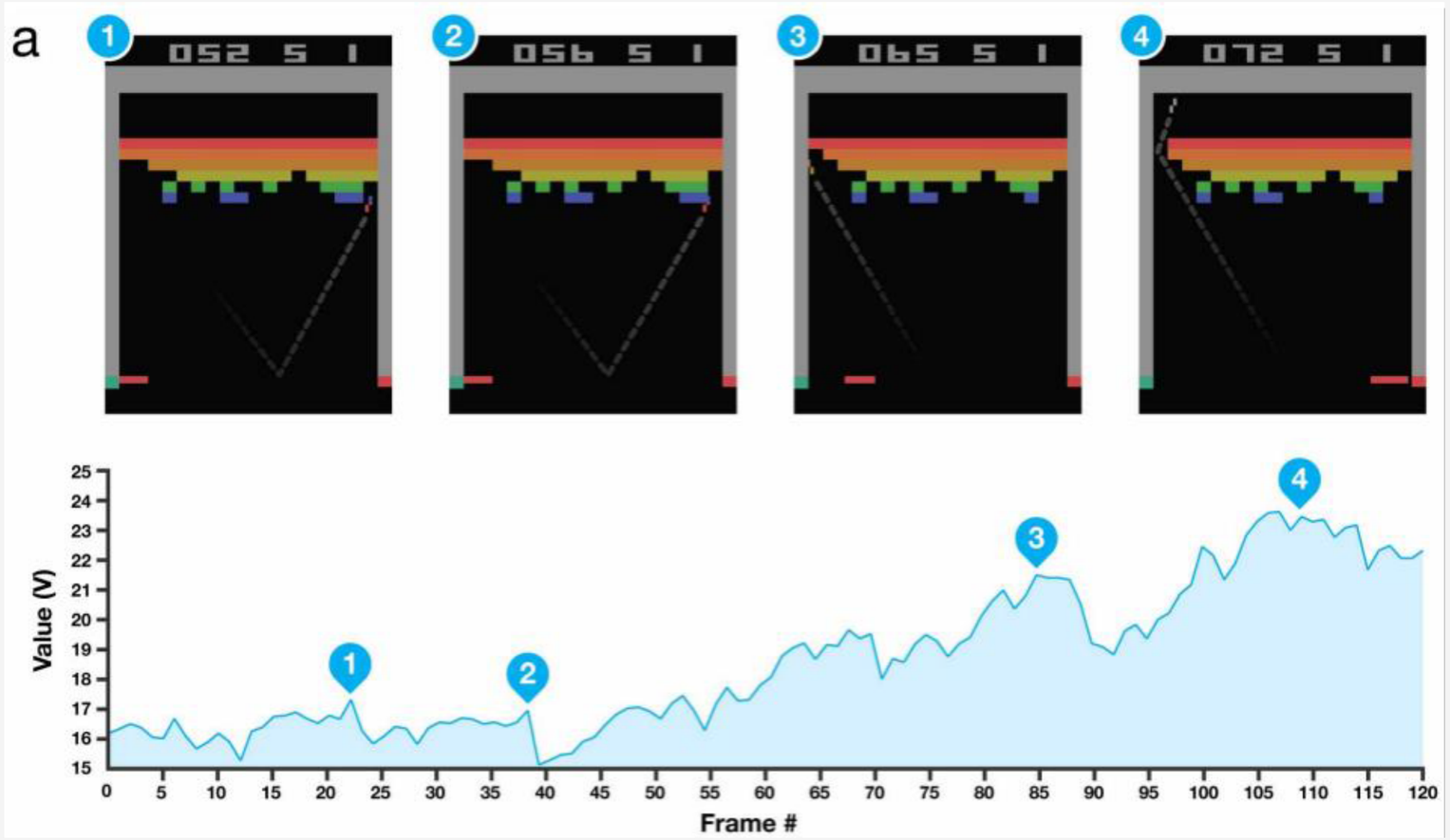
Deep Reinforcement Learning

Evolución del aprendizaje



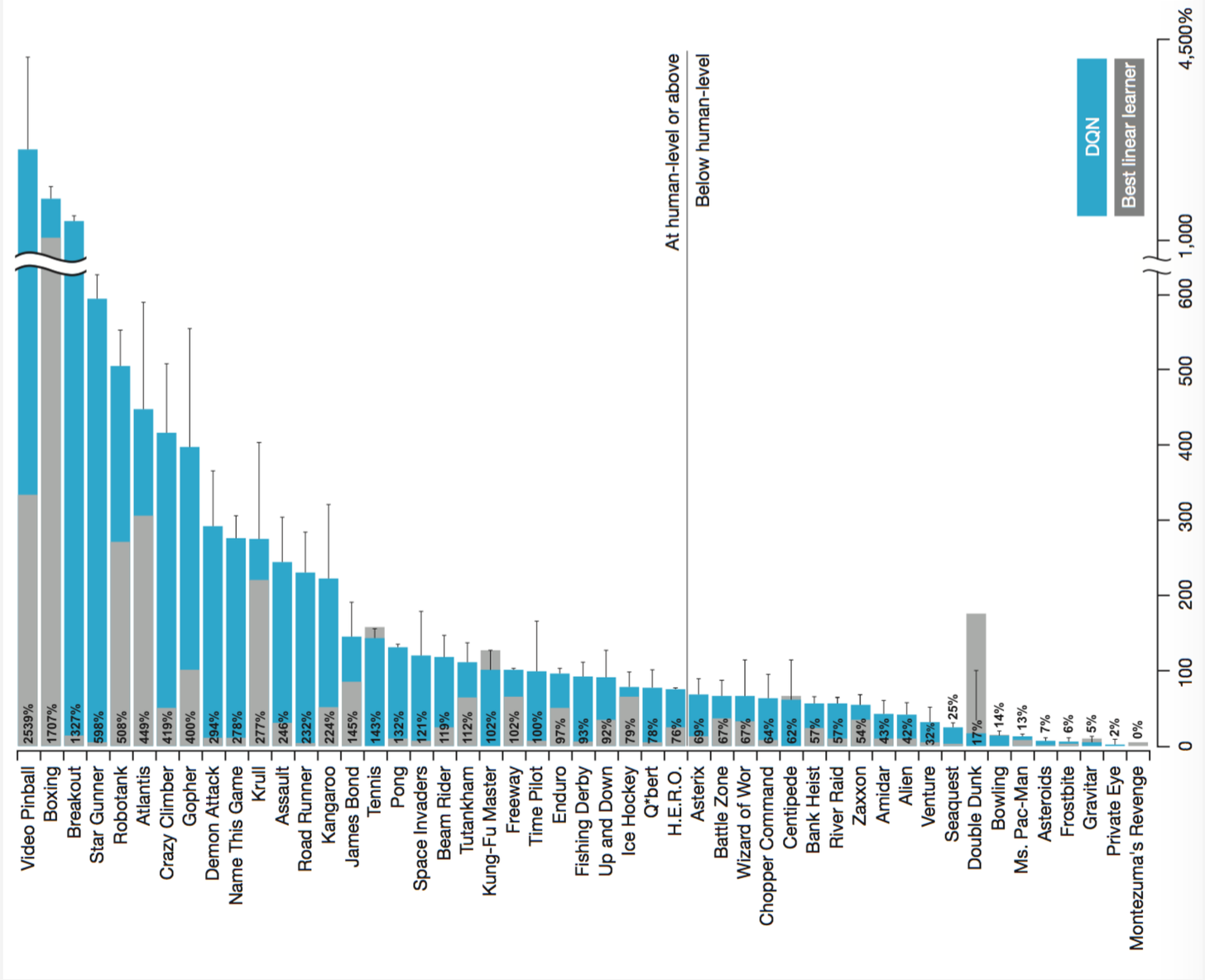
Deep Reinforcement Learning

Valoraciones aprendidas por el modelo



Deep Reinforcement Learning

El sistema bate la mejor marca humana en muchos de los juegos!



Deep Reinforcement Learning



Enduro, Atari 2600

Expert player: 368 points

Deep Learning: 661 points

Deep Reinforcement Learning



27 January 2016:

- AlphaGo, desarrollado por el equipo DeepMind, batió al campeón Go Fan Hui 5 veces de 5 en condiciones de competición

Deep Reinforcement Learning



27 January 2016:

- AlphaGo, desarrollado por el equipo DeepMind, batió al campeón Go Fan Hui 5 veces de 5 en condiciones de competición
- AlphaGo aplicaba **deep learning** mezclando aprendizaje supervisado (históricos de movimientos de expertos) y aprendizaje por refuerzo (jugando contra sí misma).

Deep Reinforcement Learning



27 January 2016:

- AlphaGo, desarrollado por el equipo DeepMind, batió al campeón Go Fan Hui 5 veces de 5 en condiciones de competición
- AlphaGo aplicaba **deep learning** mezclando aprendizaje supervisado (históricos de movimientos de expertos) y aprendizaje por refuerzo (jugando contra sí misma).

19 Octubre 2017:

- Se presenta AlphaGo Zero: aprende completamente desde cero jugando contra sí misma sin necesidad de aprendizaje supervisado: aprendizaje puramente por refuerzo. Esta version funciona mucho mejor que la anterior, llegando a batirla por 100 a 0

Retos en Deep RL

- Carga computacional: 30 imágenes de vídeo por segundo

Retos en Deep RL

- Carga computacional: 30 imágenes de vídeo por segundo
- Entrada de alta dimensión

Retos en Deep RL

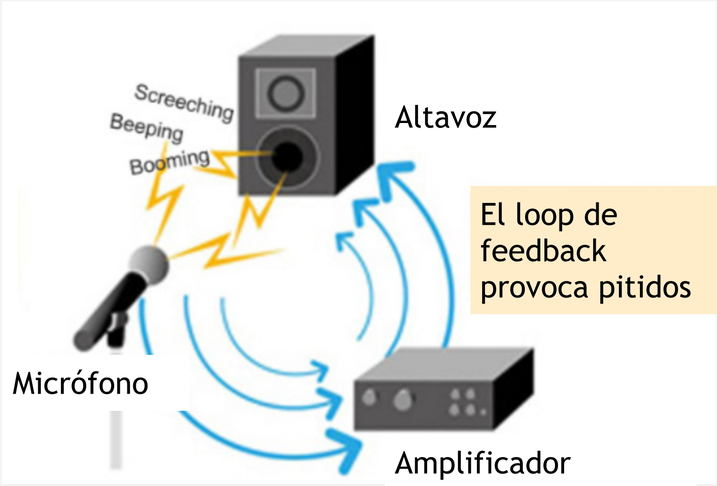
- Carga computacional: 30 imágenes de vídeo por segundo
- Entrada de alta dimensión
- No funciona con Q-learning tabular

Retos en Deep RL

- Carga computacional: 30 imágenes de vídeo por segundo
- Entrada de alta dimensión
- No funciona con Q-learning tabular
- Se sabe desde hace tiempo que la aproximación de la función de valor es teóricamente inestable

Retos en Deep RL

- Carga computacional: 30 imágenes de vídeo por segundo
- Entrada de alta dimensión
- No funciona con Q-learning tabular
- Se sabe desde hace tiempo que la aproximación de la función de valor es teóricamente inestable
- El problema es el aprendizaje a partir de la retroalimentación. Esto provoca inestabilidad en los targets a ser aprendidos



Tres problemas en Deep RL

- Cobertura

Tres problemas en Deep RL

- Cobertura
- Correlación

Tres problemas en Deep RL

- Cobertura
- Correlación
- Convergencia

Tres problemas en Deep RL

Cobertura

- La convergencia de Value Iteration, Q-learning y SARSA dependen de que, al final, se cubra todo el espacio de estados

Tres problemas en Deep RL

Cobertura

- La convergencia de Value Iteration, Q-learning y SARSA dependen de que, al final, se cubra todo el espacio de estados

- Ni siquiera cerca en problemas de alta dimensión

Tres problemas en Deep RL

Cobertura

- La convergencia de Value Iteration, Q-learning y SARSA dependen de que, al final, se cubra todo el espacio de estados
- Ni siquiera cerca en problemas de alta dimensión
- El aprendizaje humano también adolece de problemas de cobertura: el entrenamiento contra el mismo sparring conduce a un conjunto limitado de habilidades

Tres problemas en Deep RL

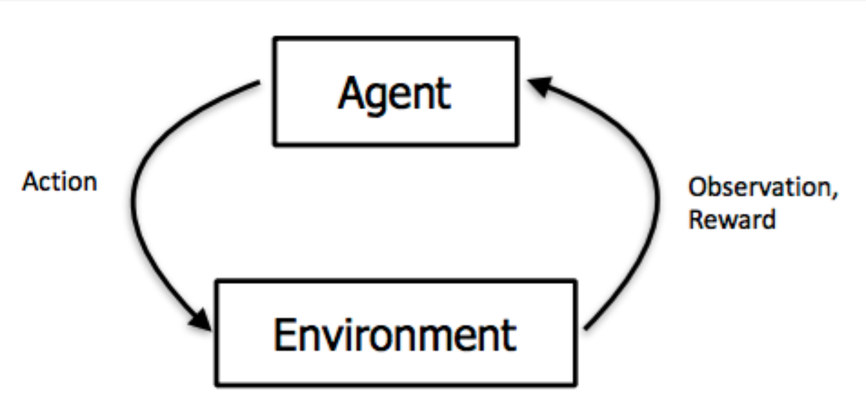
Correlación

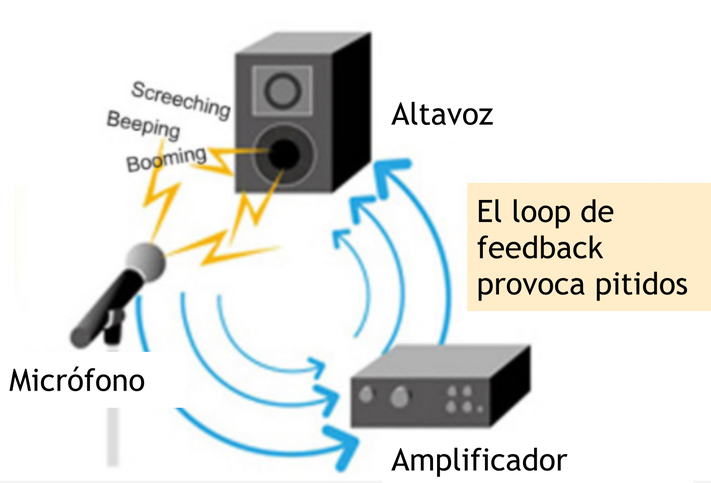
- Supervisado: los ejemplos de la base de datos no están correlacionados. Aprendizaje estable

Tres problemas en Deep RL

Correlación

- Supervisado: los ejemplos de la base de datos no están correlacionados. Aprendizaje estable
- Deep: las acciones determinan el siguiente estado del que se aprenderá





Función de coste en DQNs

Interpretación intuitiva de Q-learning

Q-learning: concepto

- Se va construyendo la tabla **Q**, que tiene tantas filas como estados posibles, y tantas columnas como acciones posibles.

Q-learning: concepto

- Se va construyendo la tabla **Q**, que tiene tantas filas como estados posibles, y tantas columnas como acciones posibles.
- Si desde el estado $\mathbf{s}_t$ se realiza una acción a_t y esta da lugar a un refuerzo $\mathbf{r}_t$ y una transición al estado $\mathbf{s}_{t+1}$:

Q-learning: concepto

- Se va construyendo la tabla **Q**, que tiene tantas filas como estados posibles, y tantas columnas como acciones posibles.
- Si desde el estado $\mathbf{s}_t$ se realiza una acción a_t y esta da lugar a un refuerzo $\mathbf{r}_t$ y una transición al estado $\mathbf{s}_{t+1}$:

$$\Delta \boldsymbol{Q}(\boldsymbol{s}_t, \boldsymbol{a}_t) = \boldsymbol{\alpha}[\boldsymbol{r}_t + \boldsymbol{\gamma}\, \boldsymbol{max}_{a'}\, \boldsymbol{Q}(\boldsymbol{s}_{t+1}, \boldsymbol{a}') - \boldsymbol{Q}(\boldsymbol{s}, \boldsymbol{a})]$$

Q-Learning: interpretación intuitiva

- Retorno total: $G_t = R_t + \gamma R_{t+1} + \cdots = \sum_{k=0}^{\infty} \gamma^k \, R_{t+k+1}$

Q-Learning: interpretación intuitiva

- Retorno total: $G_t = R_t + \gamma R_{t+1} + \cdots = \sum_{k=0}^{\infty} \gamma^k \, R_{t+k+1}$
- **Q(s,a)**: retorno total óptimo obtenido a partir del estado s si se realiza la acción **a** y a partir de ese momento se realizan las acciones óptimas

Q-Learning: interpretación intuitiva

- Retorno total: $G_t = R_t + \gamma R_{t+1} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$
- **Q(s,a)**: retorno total óptimo obtenido a partir del estado s si se realiza la acción **a** y a partir de ese momento se realizan las acciones óptimas
- Cuando las **Q** convergen a los valores óptimos, se debe cumplir (Bellman):

$Q(s_t, a) = r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$

Q-Learning: interpretación intuitiva

- Retorno total: $G_t = R_t + \gamma R_{t+1} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$
- **Q(s,a)**: retorno total óptimo obtenido a partir del estado s si se realiza la acción **a** y a partir de ese momento se realizan las acciones óptimas
- Cuando las **Q** convergen a los valores óptimos, se debe cumplir (Bellman):

  $Q(s_t,a) = r + \gamma \cdot \max a' \; Q(s_{t+1}, a')$

- Si la parte de la **derecha** es mayor (menor) que la de la **izquierda**, $Q(s_t,a)$ debe aumentar (disminuir):

  $\Delta Q(s(t),a) = \alpha \cdot [\, r + \gamma \cdot \max a' \; Q(s_{t+1}, a') \; - \; Q(s_t,a) \,]$

Problemas de Q-Learning

Problemas

- En problemas reales (número de estados) x (acciones posibles) es un número enorme (o infinito).

Tabla Q

	acción 1	acción 2	...	acción m
estado 1	$Q(s_1, a_1)$	$Q(s_1, a_2)$	...	$Q(s_1, a_m)$
estado 2	$Q(s_2, a_1)$	$Q(s_2, a_2)$	...	$Q(s_2, a_m)$
...	...	...	...	...
estado N	$Q(s_N, a_1)$	$Q(s_N, a_2)$	...	$Q(s_N, a_m)$

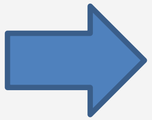
Problemas de Q-Learning

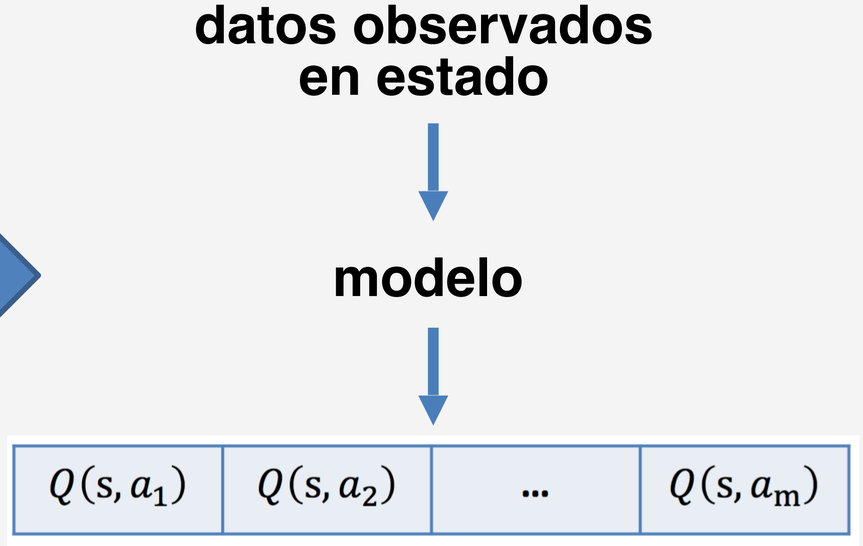
Problemas

- En problemas reales (número de estados) x (acciones posibles) es un número enorme (o infinito).
- **Solución:** sustituir la tabla por un modelo (por ejemplo una red neuronal con muchos menos parámetros)

Tabla Q

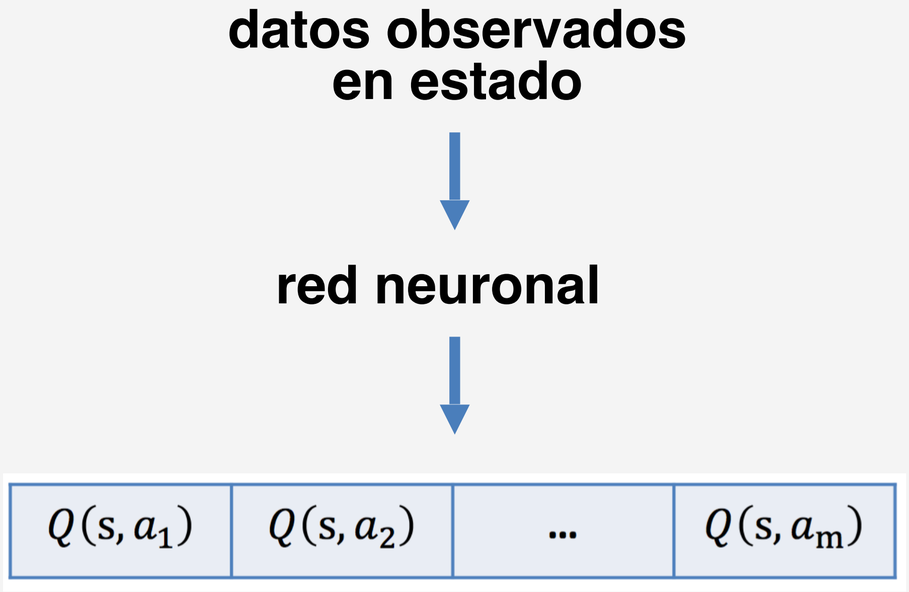
	acción 1	acción 2	...	acción m
estado 1	$Q(s_1, a_1)$	$Q(s_1, a_2)$	...	$Q(s_1, a_m)$
estado 2	$Q(s_2, a_1)$	$Q(s_2, a_2)$	...	$Q(s_2, a_m)$
...	...	...	...	...
estado N	$Q(s_N, a_1)$	$Q(s_N, a_2)$	...	$Q(s_N, a_m)$



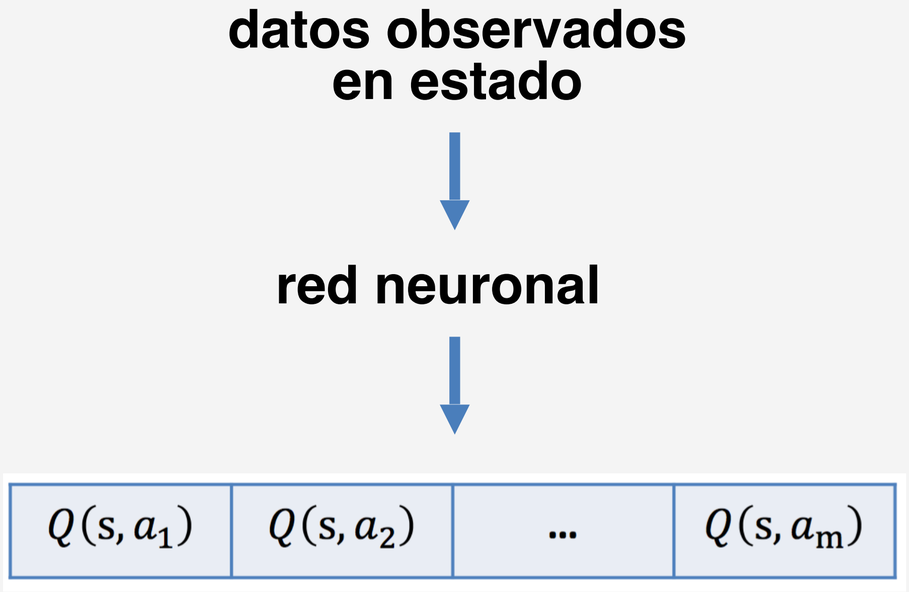


DQN: función de coste

DQN



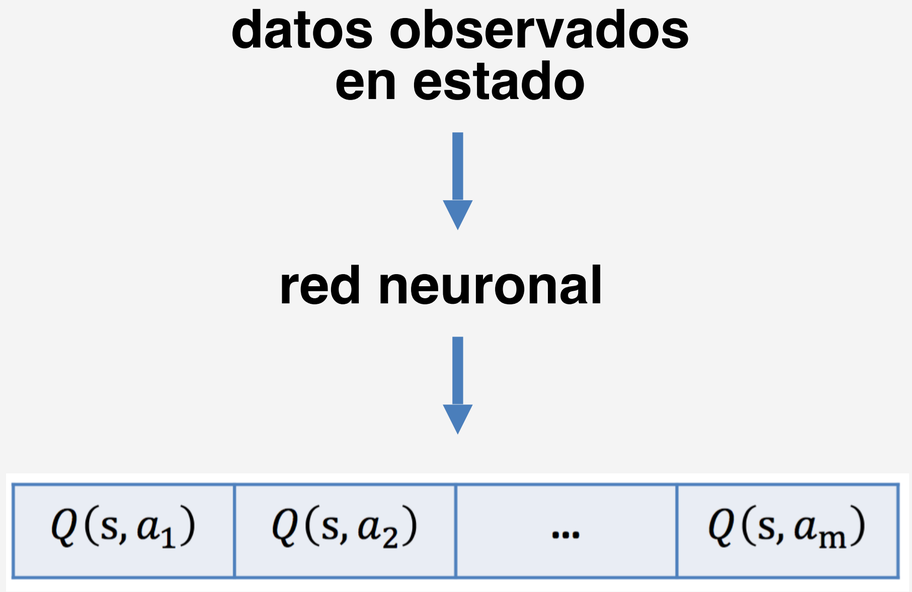
DQN



Cuando las **Q** convergen a los valores óptimos, se debe cumplir (Bellman):

$$\underbrace{Q(s_t, a)}_{\text{Predicción}} = \underbrace{r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')}_{\text{Target}}$$

DQN



Cuando las **Q** convergen a los valores óptimos, se debe cumplir (Bellman):

$$\underbrace{Q(s_t, a)}_{\text{Predicción}} = \underbrace{r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')}_{\text{Target}}$$

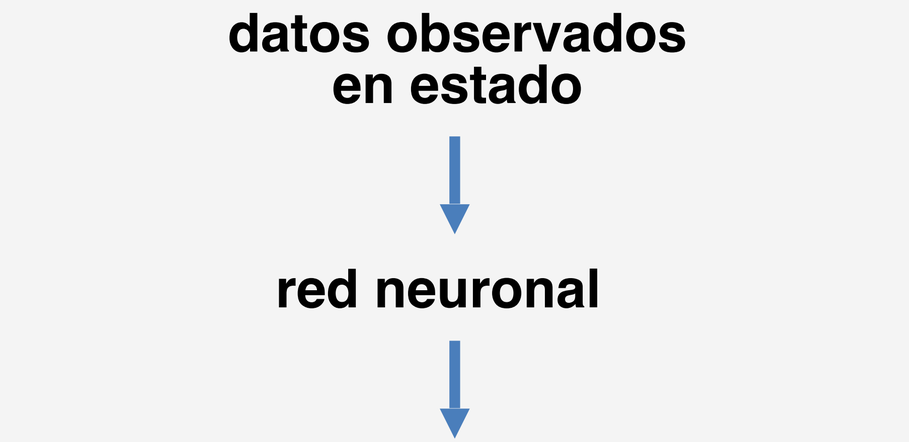
$$loss = \left(\underbrace{r_t + \gamma \max_{a`} \hat{Q}(s_{t+1}, a`)}_{\text{Target}} - \underbrace{Q(s_t, a_t)}_{\text{Prediction}} \right)^2$$

Reward

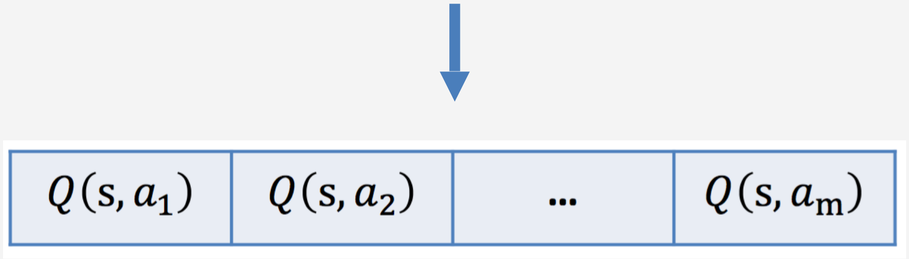
Decay Rate

DQN

datos observados en estado



red neuronal



- **Remember**
- **Replay**

Cuando las **Q** convergen a los valores óptimos, se debe cumplir (Bellman):

$$\underbrace{Q(s_t, a)}_{\text{Predicción}} = \underbrace{r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')}_{\text{Target}}$$

$$loss = \left(\underbrace{r_t + \gamma \max_{a`} \hat{Q}(s_{t+1}, a`)}_{\text{Target}} - \underbrace{Q(s_t, a_t)}_{\text{Prediction}} \right)^2$$

Reward

Decay Rate

DQN

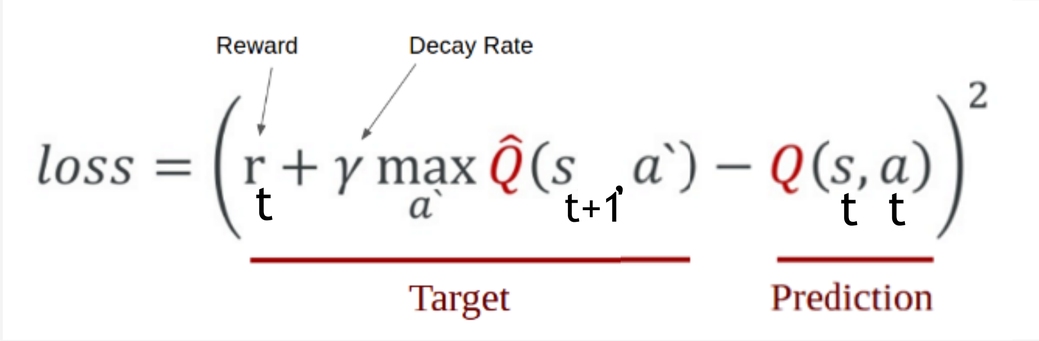
- Se sustituye la tabla Q por una red neuronal que tome como entrada los datos observados en el estado actual

DQN

- Se sustituye la tabla Q por una red neuronal que tome como entrada los datos observados en el estado actual
- La capa de salida tiene tantas neuronas como posibles acciones

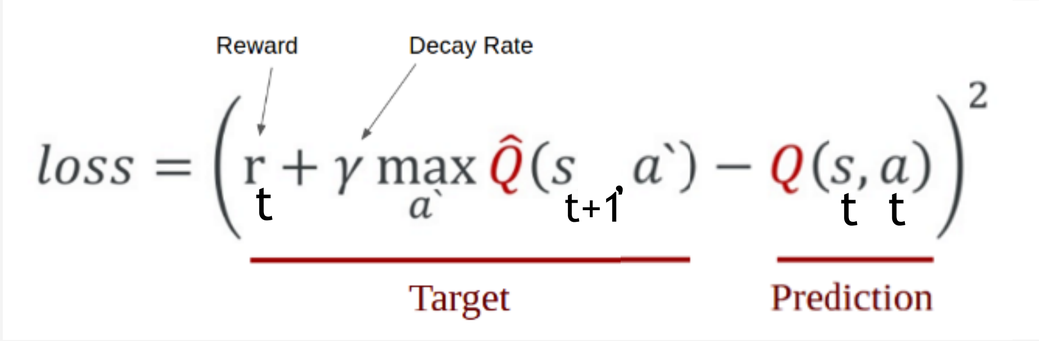
DQN

- Se sustituye la tabla Q por una red neuronal que tome como entrada los datos observados en el estado actual
- La capa de salida tiene tantas neuronas como posibles acciones
- La función de activación de las neuronas de esta última capa debe ser la identidad (función “lineal”) ya que el problema es de regresión



DQN

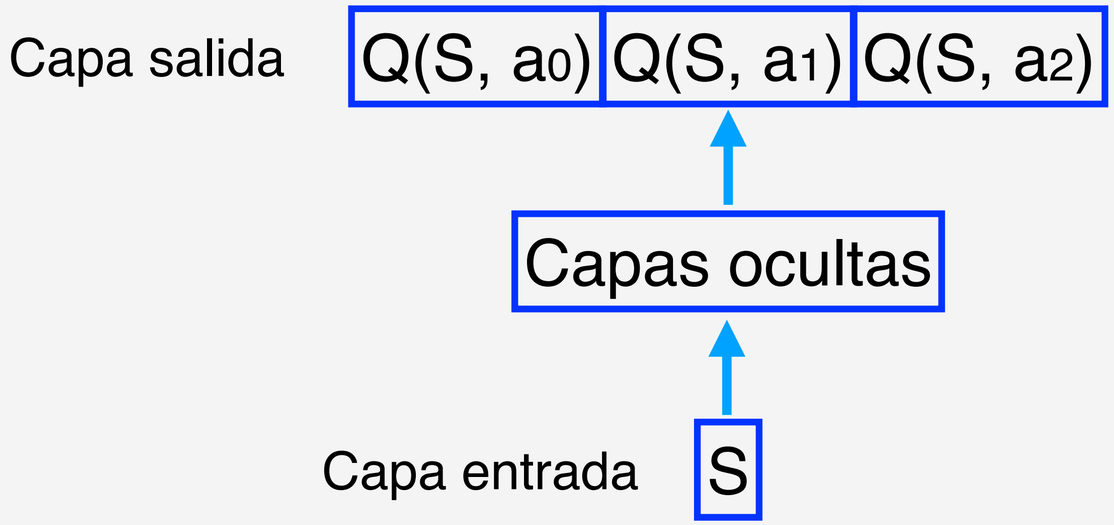
- Se sustituye la tabla Q por una red neuronal que tome como entrada los datos observados en el estado actual
- La capa de salida tiene tantas neuronas como posibles acciones
- La función de activación de las neuronas de esta última capa debe ser la identidad (función “lineal”) ya que el problema es de regresión



- Cada término de error está asociado únicamente a una neurona de salida concreta (la predicción sobre la acción realizada)

DQN: ejemplo

- Supongamos que tenemos un problema con 3 posibles acciones
- La red entonces tiene 3 neuronas de salida:



DQN: ejemplo

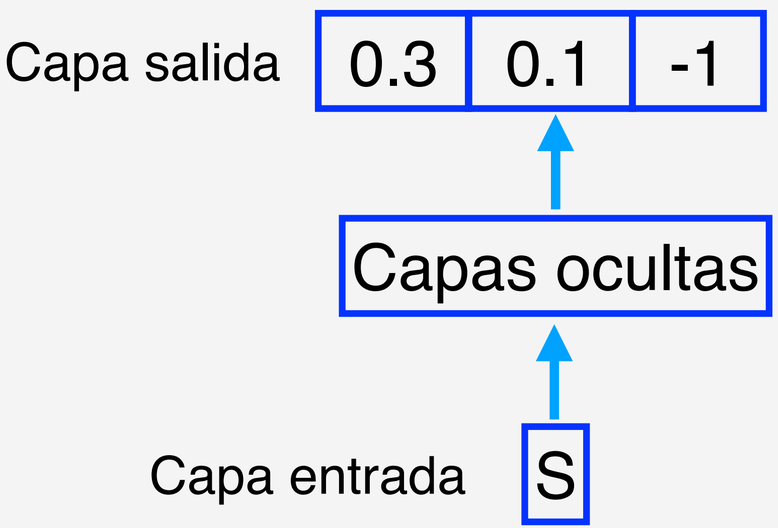
- Supongamos que el agente ha pasado del estado S al S’ mediante la acción 0

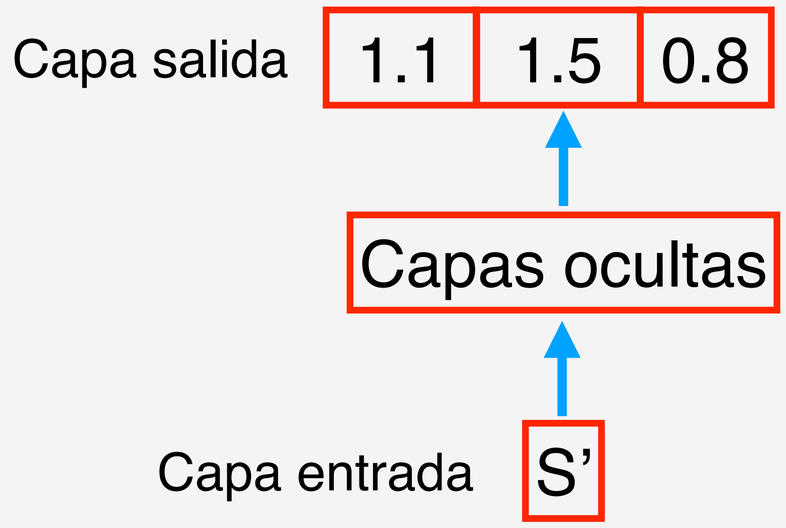
DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S’ mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9

DQN: ejemplo

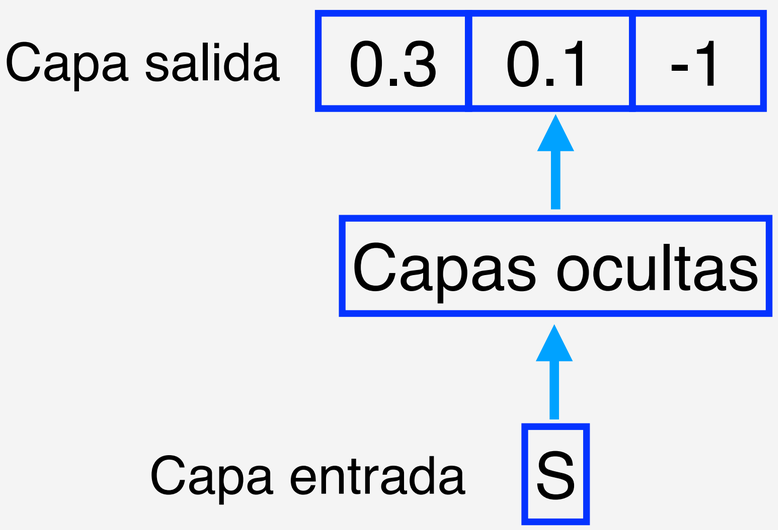
- Supongamos que el agente ha pasado del estado S al S’ mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S’ es:

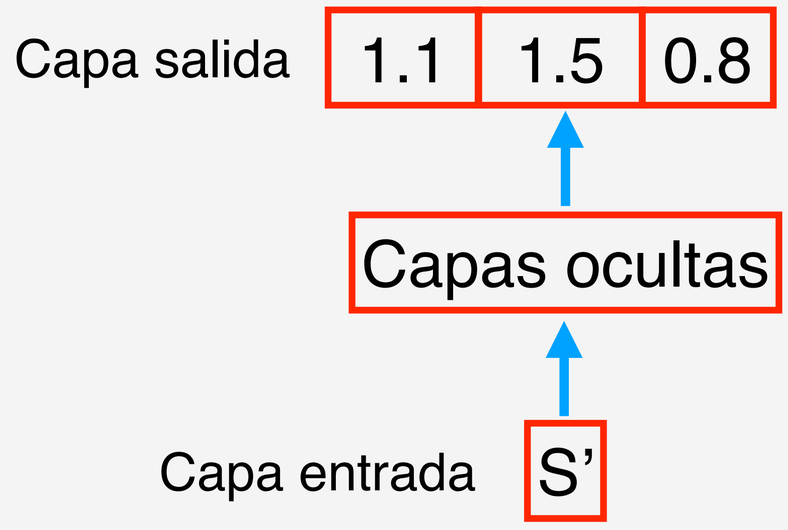




DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S' mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S' es:



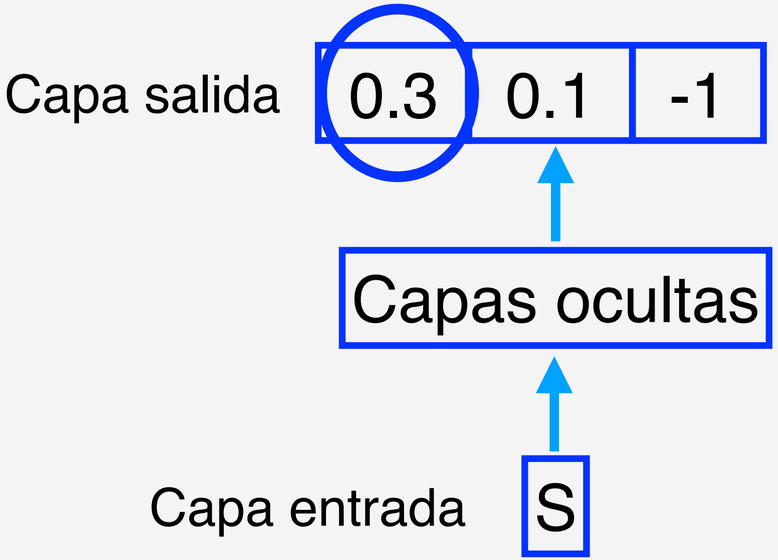


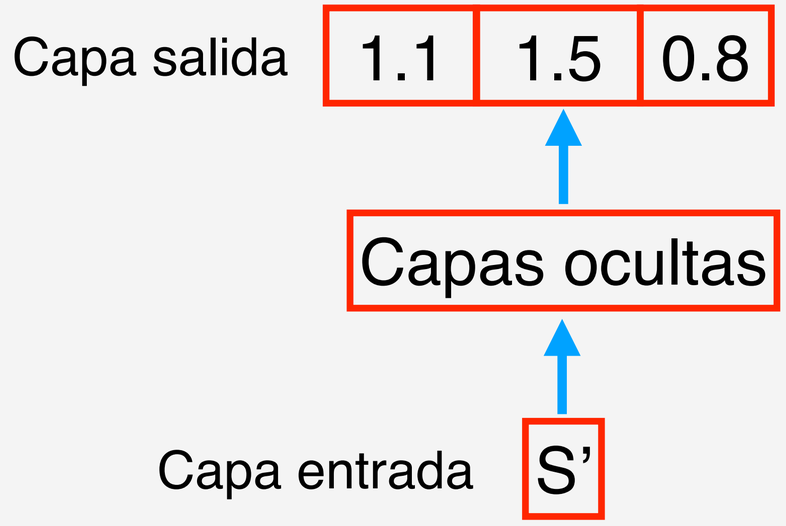
- De acuerdo a Q-learning, tendríamos:

$$\Delta Q(s_t, a_t) = \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S' mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S' es:



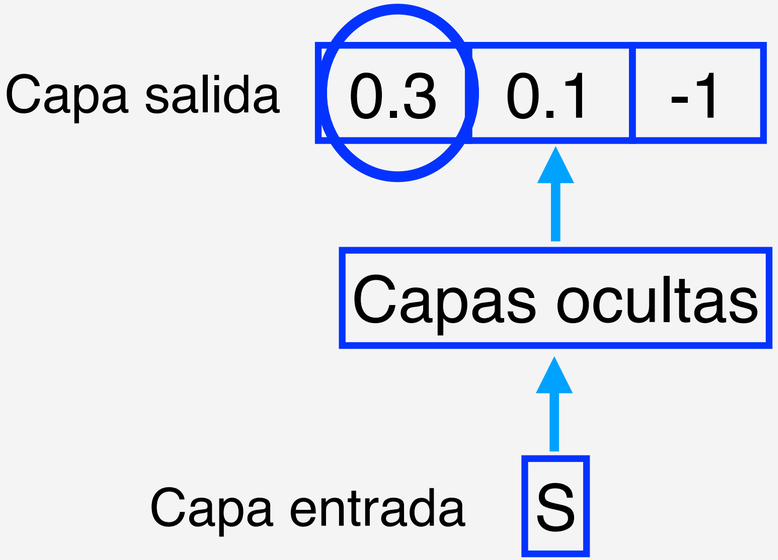


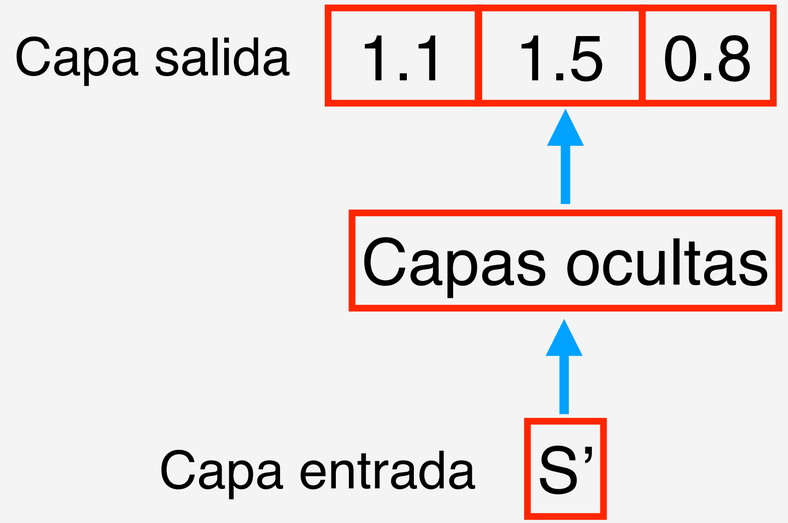
- De acuerdo a Q-learning, tendríamos:

$$\Delta Q(s_t, a_t) = \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S' mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S' es:



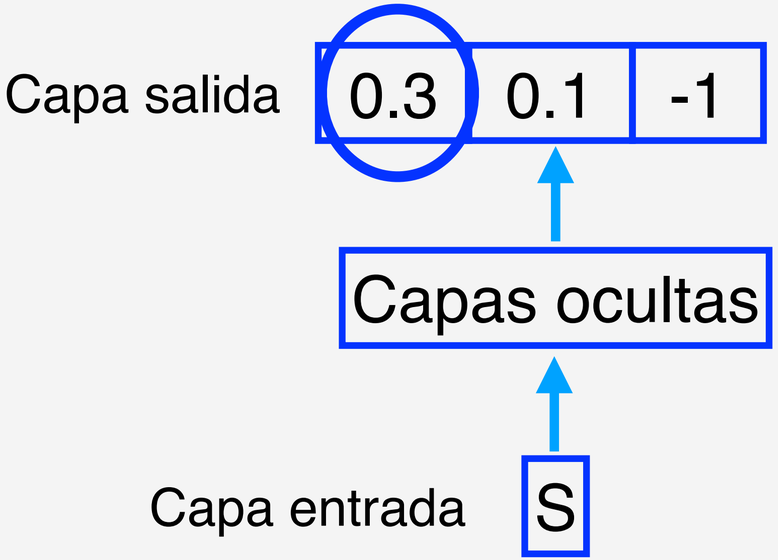


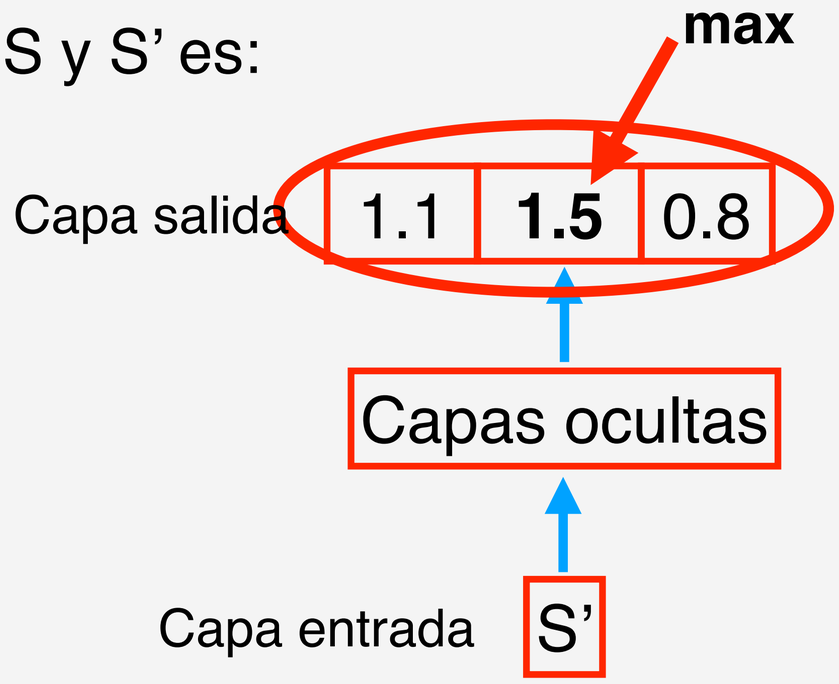
- De acuerdo a Q-learning, tendríamos:

$$\Delta Q(s_t, a_t) = \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S' mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S' es:



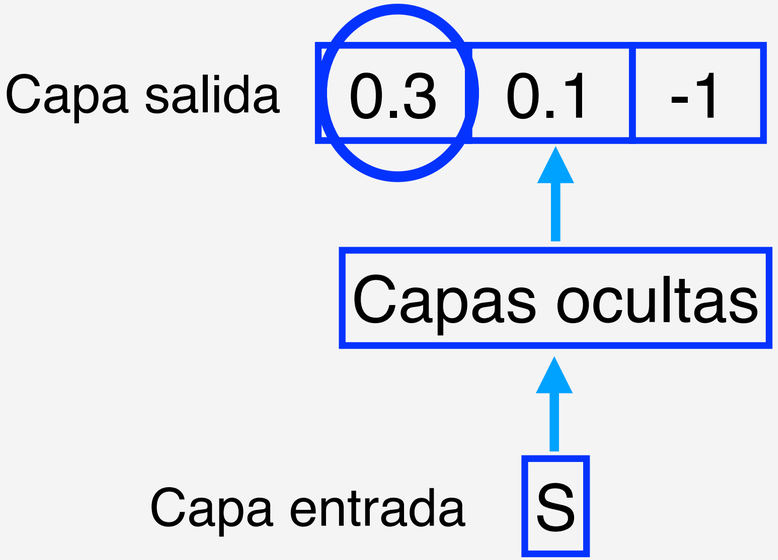


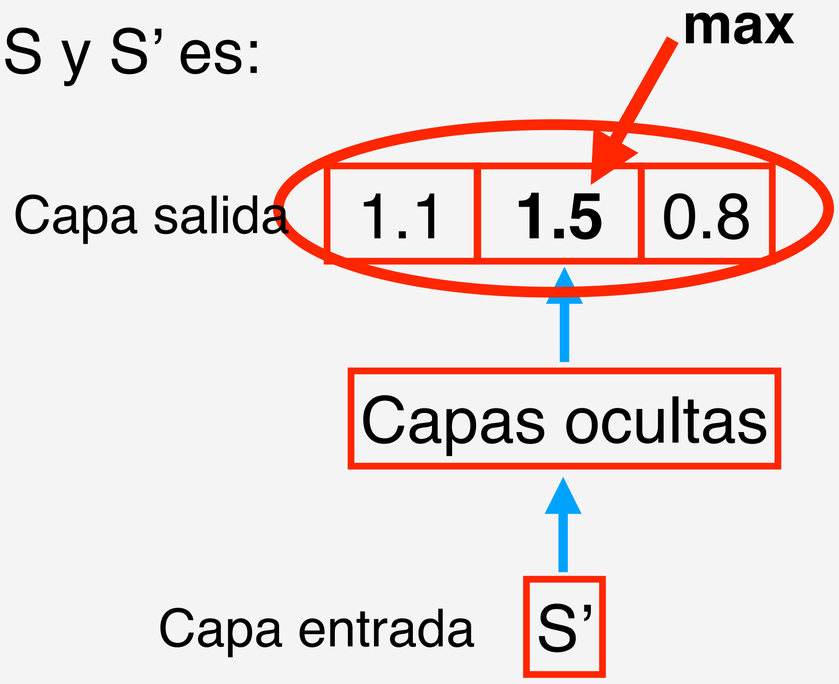
- De acuerdo a Q-learning, tendríamos:

$$\Delta Q(s_t, a_t) = \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S' mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S' es:



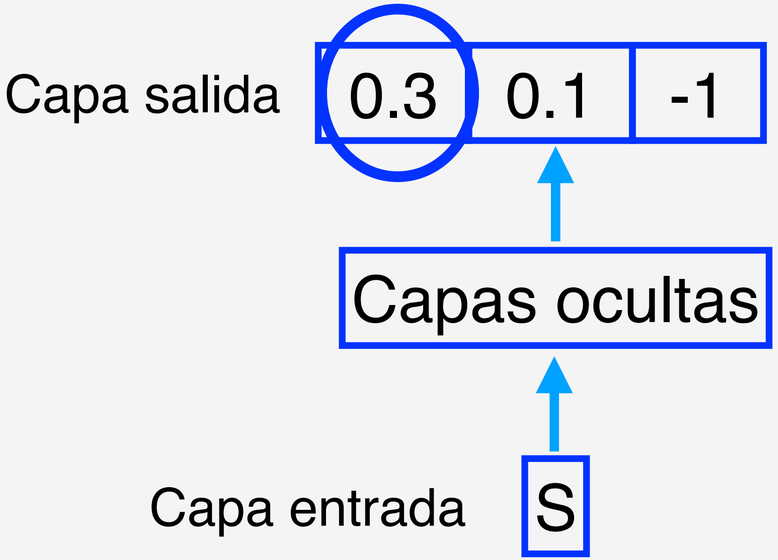


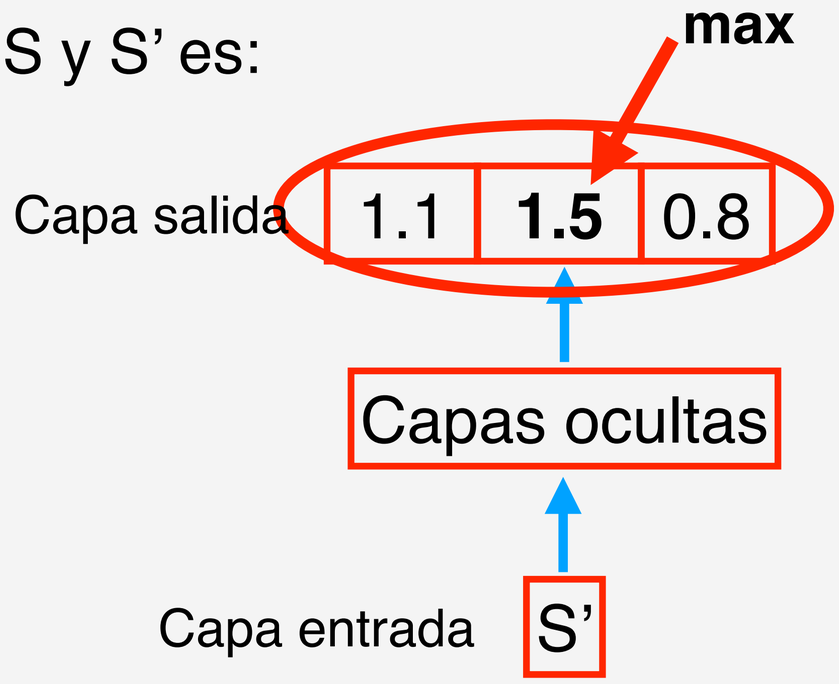
- De acuerdo a Q-learning, tendríamos:

$$\Delta Q(s_t, a_t) = \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S' mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S' es:



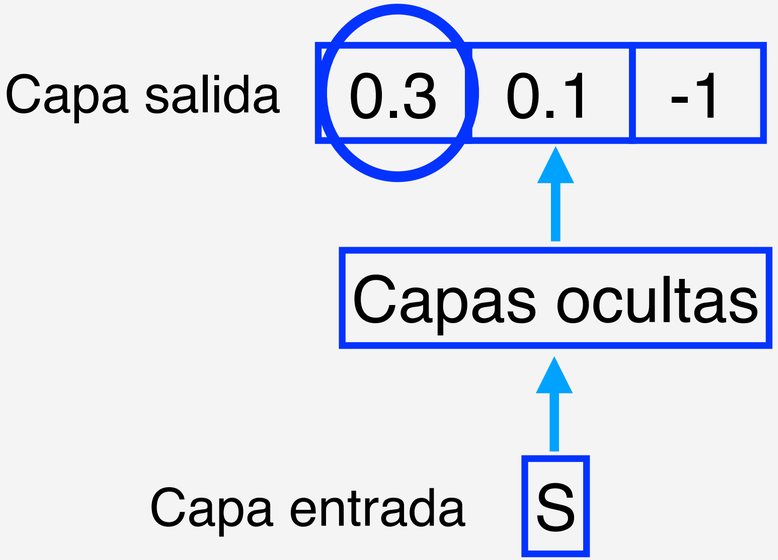


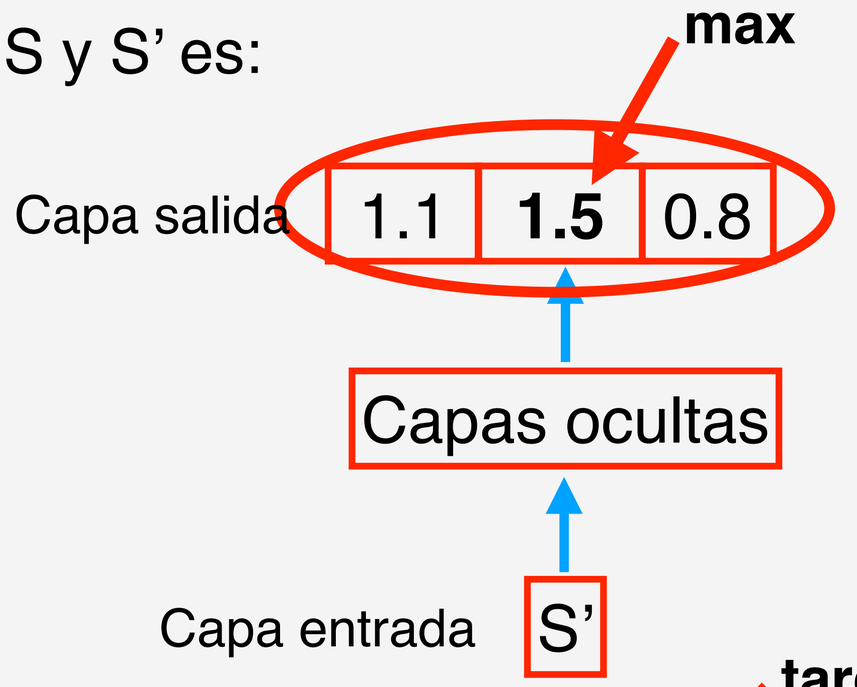
- De acuerdo a Q-learning, tendríamos:

$$\Delta Q(s_t, a_t) = \alpha [2 + 0.9 \mathbf{1.5} - 0.3] = \alpha [3.35 - 0.3]$$

DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S' mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S' es:





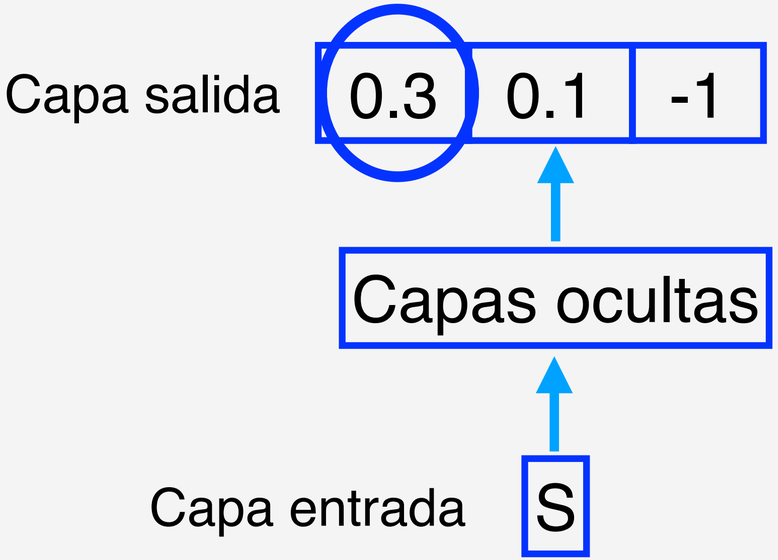
- De acuerdo a Q-learning, tendríamos:

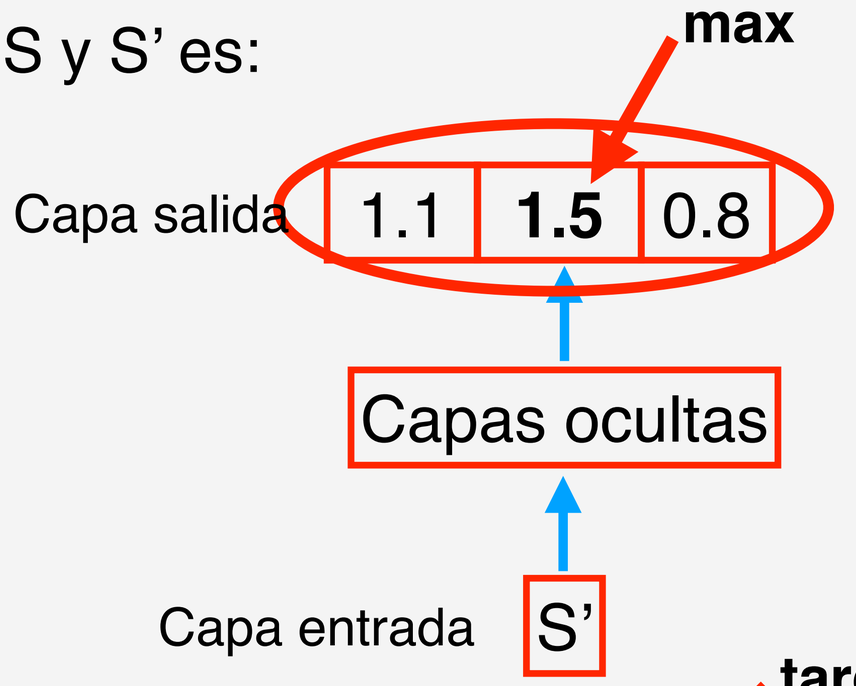
$$\Delta Q(s_t, a_t) = \alpha [2 + 0.9 \mathbf{1.5} - 0.3] = \alpha [3.35 - 0.3]$$

target

DQN: ejemplo

- Supongamos que el agente ha pasado del estado S al S' mediante la acción 0
- Se ha obtenido refuerzo 2 y el γ es 0.9
- Supongamos que la salida de la red para S y S' es:





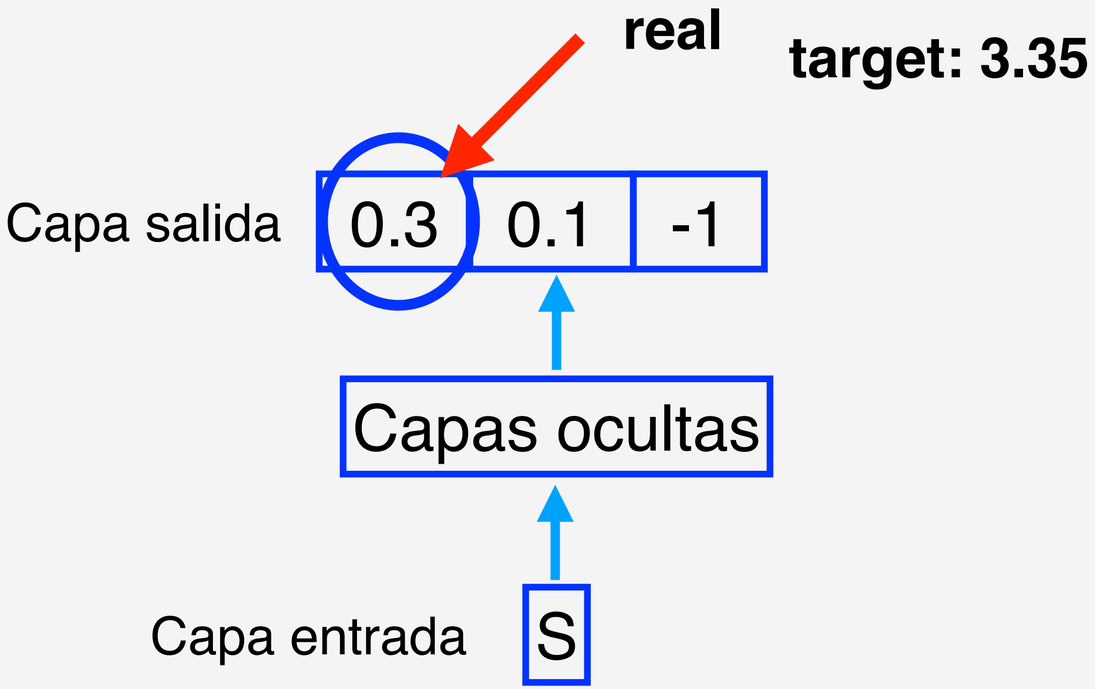
- De acuerdo a Q-learning, tendríamos:

$$\Delta Q(s_t, a_t) = \alpha [2 + 0.9\ \mathbf{1.5} - 0.3] = \alpha [3.35 - 0.3]$$

target **predicción**

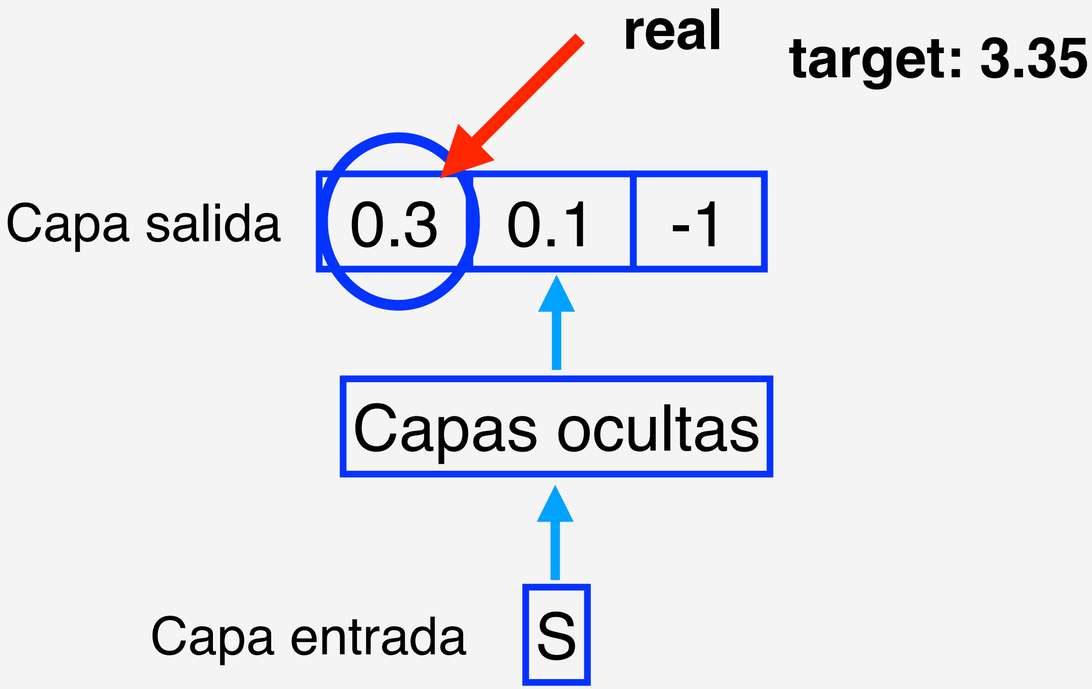
DQN: ejemplo

- Resumiendo, tenemos que cambiar la valoración de la acción 0 (la realizada) desde el estado S:



DQN: ejemplo

- Resumiendo, tenemos que cambiar la valoración de la acción 0 (la realizada) desde el estado S:

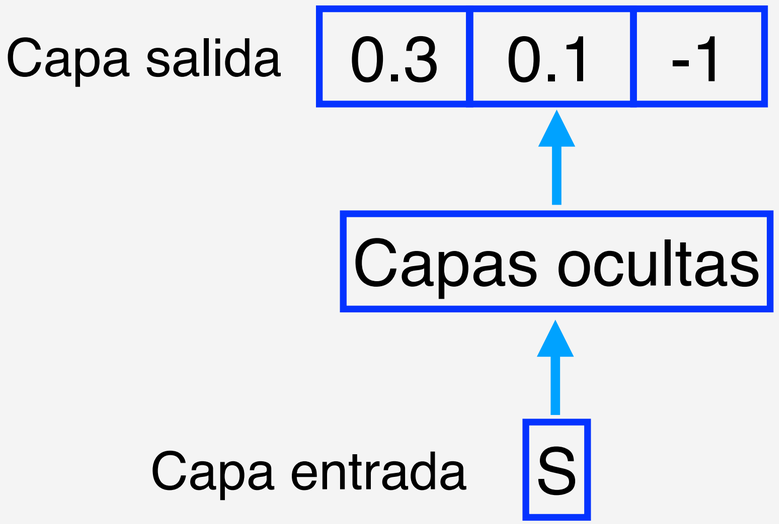


- El resto de valoraciones de S están ok de momento

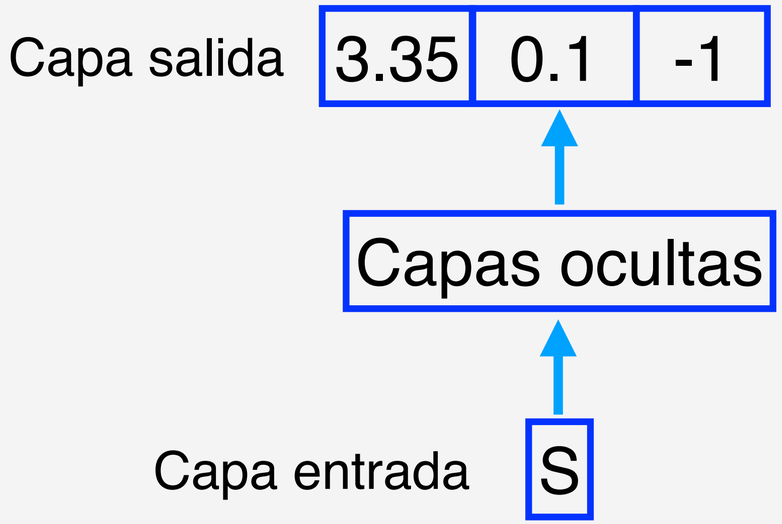
DQN: ejemplo

- Resumiendo, tenemos que cambiar la valoración de la acción 0 (la realizada) desde el estado S:

salida para S:



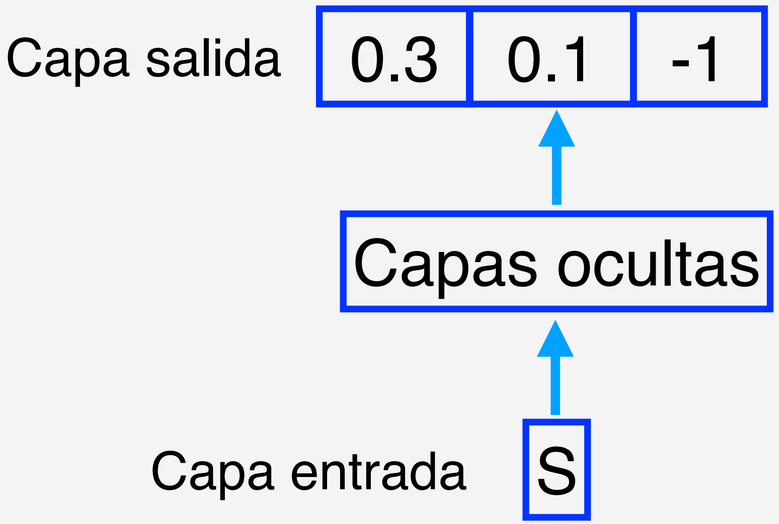
salida deseada para S:



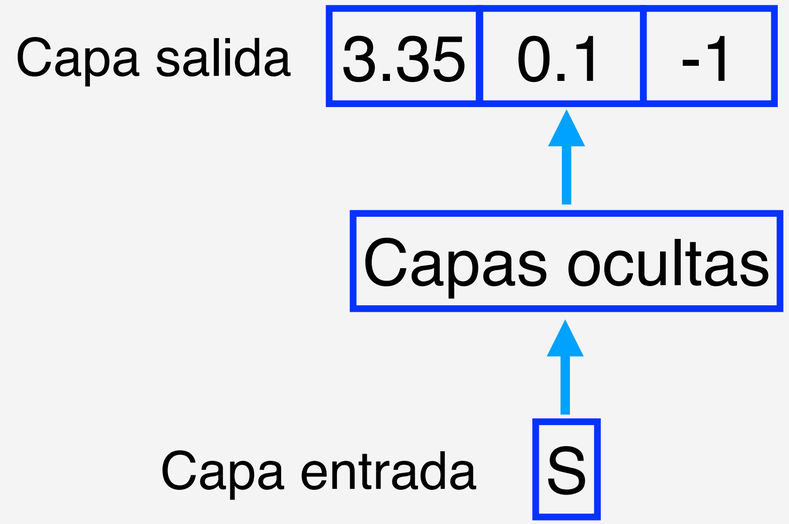
DQN: ejemplo

- Resumiendo, tenemos que cambiar la valoración de la acción 0 (la realizada) desde el estado S:

salida para S:



salida deseada para S:



$$loss = \left(\underbrace{r_t + \gamma \max_{a`} \hat{Q}(s_{t+1}, a`)}_{\text{Target}} - \underbrace{Q(s_t, a_t)}_{\text{Prediction}} \right)^2$$

Reward

Decay Rate

Q-learning vs. DQN

Q-learning vs. DQN

	Q-learning	**DQN**
Estructura	Tabla	Red neuronal
Información:	En celdas	En pesos
Estados	Conjunto finito	Infinitos
Se actualiza:	Una celda Q(s, a)	Los pesos de la red
La actualización afecta en cada iteración a:	Una celda	Muchos estados

Conclusiones

Función de coste DQN: conclusiones

- El problema de optimización en DQN es una especie de problema de regresión

Función de coste DQN: conclusiones

- El problema de optimización en DQN es una especie de problema de regresión
- Pero los targets son dinámicos: van cambiando.

Función de coste DQN: conclusiones

- El problema de optimización en DQN es una especie de problema de regresión
- Pero los targets son dinámicos: van cambiando.
- Los targets dependen de la misma red!

Función de coste DQN: conclusiones

- El problema de optimización en DQN es una especie de problema de regresión
- Pero los targets son dinámicos: van cambiando.
- Los targets dependen de la misma red!
- Normalmente se usa como función de coste el MSE

Función de coste DQN: conclusiones

- El problema de optimización en DQN es una especie de problema de regresión
- Pero los targets son dinámicos: van cambiando.
- Los targets dependen de la misma red!
- Normalmente se usa como función de coste el MSE
- Para cada transición **S, a, r, S'** el target deseado es la salida que ha realizado la red para S, salvo el de la neurona correspondiente a la valoración de la acción que se ha realizado, que hay que actualizar

Entrenamiento de una DQN

Algoritmo básico de entrenamiento de una DQN

Algoritmo básico de DQN

1. Inicializar pesos aleatoriamente

Algoritmo básico de DQN

1. Inicializar pesos aleatoriamente

2. Inicializar episodio e iterar hasta su fin:
   - Generar experiencia S, a, r, S’ y meterla en memoria limitada (“buffer”)
   - Tomar de buffer minibatch de experiencias y entrenar la red con ella

Algoritmo básico de DQN

1. Inicializar pesos aleatoriamente

2. Inicializar episodio e iterar hasta su fin:
   - Generar experiencia S, a, r, S’ y meterla en memoria limitada (“buffer”)
   - Tomar de buffer minibatch de experiencias y entrenar la red con ella

3. Volver a 2 hasta fin por número de episodios o convergencia

Algoritmo básico de DQN

- Entrenar con minibatches de experiencias elegidas al azar actúa contra el problema de la correlación de experiencias

Algoritmo básico de DQN

- Entrenar con minibatches de experiencias elegidas al azar actúa contra el problema de la correlación de experiencias

- Si nos enfocamos en reentrenar la red con las últimas experiencias, “olvida” las iniciales

Algoritmo básico de DQN

- Entrenar con minibatches de experiencias elegidas al azar actúa contra el problema de la correlación de experiencias

- Si nos enfocamos en reentrenar la red con las últimas experiencias, “olvida” las iniciales

- “Experience replay”: las redes neuronales necesitan que les “recuerden” las experiencias pasadas

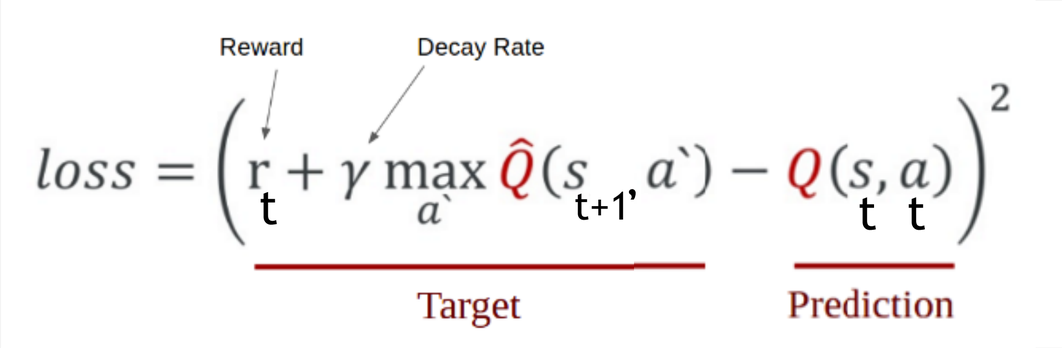
Algunas mejoras

Algunas mejoras en DQN

- Cuidado con refuerzos muy diferentes (Atari): por ejemplo, hacer clip entre -1 y 1

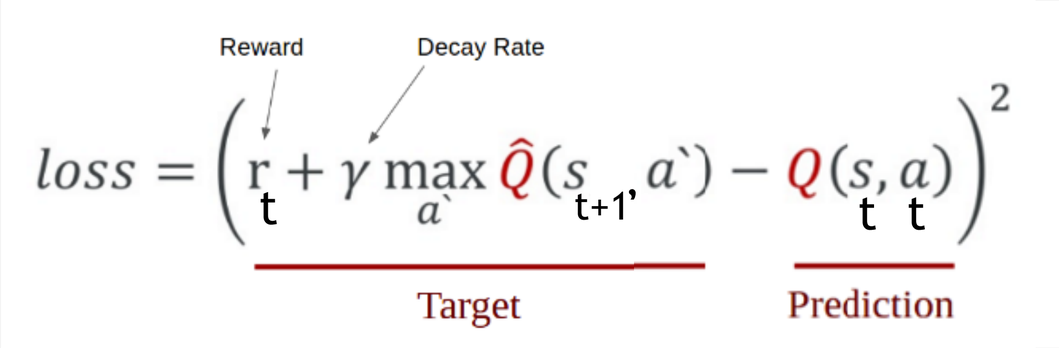
Algunas mejoras en DQN

- Cuidado con refuerzos muy diferentes (Atari): por ejemplo, hacer clip entre -1 y 1
- A veces el aprendizaje es inestable (el target está dado por la misma red):



Algunas mejoras en DQN

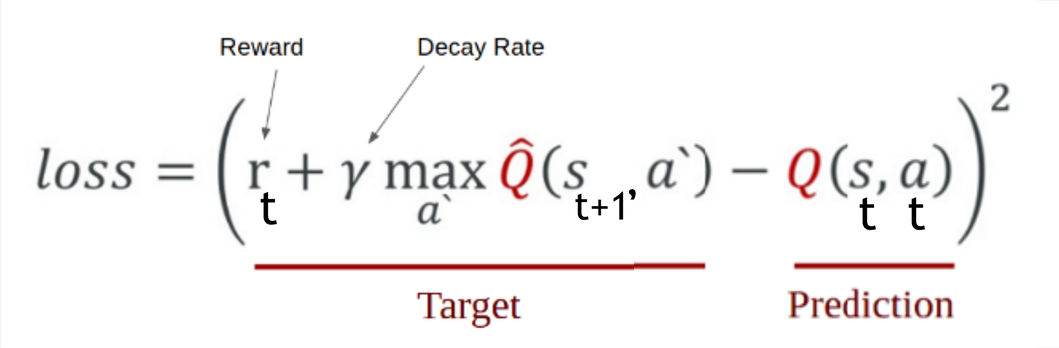
- Cuidado con refuerzos muy diferentes (Atari): por ejemplo, hacer clip entre -1 y 1
- A veces el aprendizaje es inestable (el target está dado por la misma red):



- Para estabilizar el aprendizaje se puede utilizar una red neuronal Q para generar la política y otra Q’ para calcular los targets

Algunas mejoras en DQN

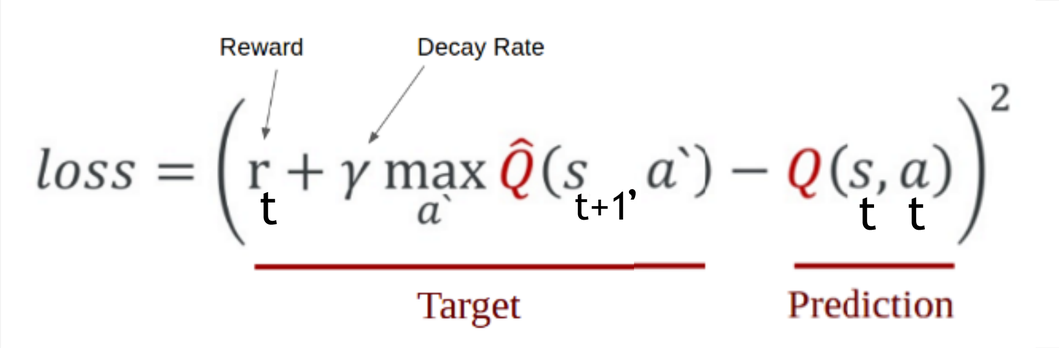
- Cuidado con refuerzos muy diferentes (Atari): por ejemplo, hacer clip entre -1 y 1
- A veces el aprendizaje es inestable (el target está dado por la misma red):



- Para estabilizar el aprendizaje se puede utilizar una red neuronal Q para generar la política y otra Q’ para calcular los targets
- Q es la que se va modificando paso a paso

Algunas mejoras en DQN

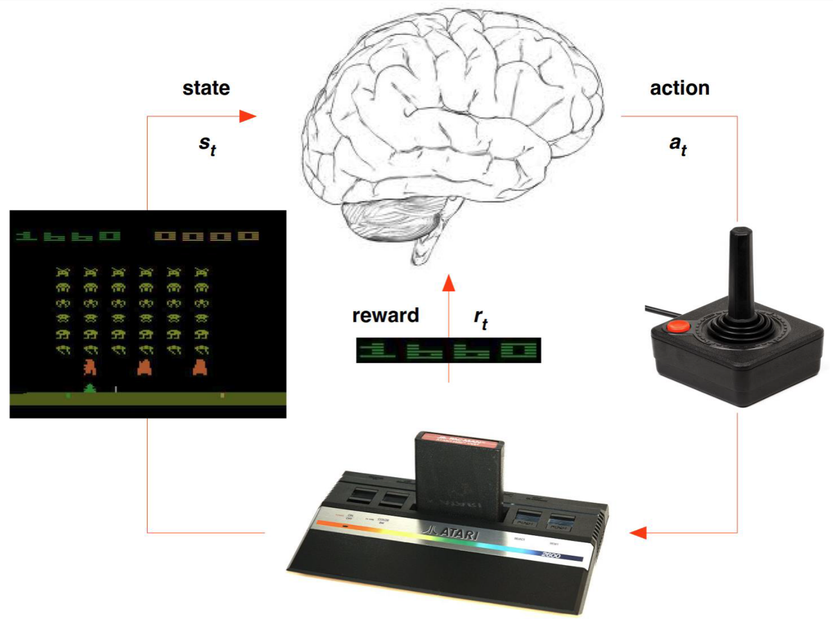
- Cuidado con refuerzos muy diferentes (Atari): por ejemplo, hacer clip entre -1 y 1
- A veces el aprendizaje es inestable (el target está dado por la misma red):



- Para estabilizar el aprendizaje se puede utilizar una red neuronal Q para generar la política y otra Q’ para calcular los targets
- Q es la que se va modificando paso a paso
- Cada k iteraciones se copia la Q’ en Q

Algunas mejoras en DQN

Algoritmo DQN Atari



Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$

For episode $= 1, M$ **do**

  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

  For $t = 1$,T **do**

    With probability ε select a random action a_t

    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$

    Execute action a_t in emulator and observe reward r_t and image x_{t+1}

    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^{-}) & \text{otherwise} \end{cases}$

    Perform a gradient descent step on $\left(y_j - Q(\phi_j, a_j; \theta)\right)^2$ with respect to the network parameters θ

    Every C steps reset $\hat{Q} = Q$

  End For

End For